

Joc del Pingu

Guia d'usuari i guia tècnica completes

Document generat automàticament des de l'esquema d'Obsidian

Índex

1. 00 - Inici
2. Guia d'Usuari / Visió General
3. Guia d'Usuari / Menú Principal
4. Guia d'Usuari / Configuració de Partida
5. Guia d'Usuari / Tauler de Joc
6. Guia d'Usuari / Caselles Especials
7. Guia d'Usuari / Inventari i Objectes
8. Guia d'Usuari / Guardar i Carregar
9. Guia d'Usuari / Mode Debug
10. Guia d'Usuari / Idiomes i Localització
11. Guia Tècnica / Arquitectura General
12. Guia Tècnica / Estructura de Paquets
13. Guia Tècnica / Paquet model.board
14. Guia Tècnica / Paquet model.entity
15. Guia Tècnica / Paquet model.item
16. Guia Tècnica / Paquet model.game
17. Guia Tècnica / Paquet model.config
18. Guia Tècnica / Paquet model.db
19. Guia Tècnica / Paquet controller
20. Guia Tècnica / Paquet view
21. Guia Tècnica / Sistema de Sprites
22. Guia Tècnica / Sistema de So
23. Guia Tècnica / Sistema d'Idiomes
24. Guia Tècnica / Sistema de Persistència
25. Guia Tècnica / Diagrama de Classes
26. Guia Tècnica / Flux de Joc
27. Guia Tècnica / Mode Debug Tècnic

00 - Inici · El Joc del Pingüí

BENVINGUDA

Aquest vault d'Obsidian documenta el projecte **Joc del Pingü** (Joc del Pingüí), un joc de taula tipus *snake-board* desenvolupat en **Java + JavaFX** com a pràctica del curs de Desenvolupament d'Aplicacions Multiplataforma.

Què és aquest joc?

El Joc del Pingüí és un joc de **2-4 jugadors per torns** ambientat a l'Àrtic. Els jugadors mouen pingüins al llarg d'un tauler de 50 caselles en forma de serp tirant daus, i han de superar perills (forats al gel, ós polar, terra trencat...) per arribar primers al final. Una **foca CPU** opcional fa de rival comú que pot fer perdre la partida a tothom si arriba al final abans.

Per on començar

Aquesta documentació té dues guies independents segons el teu perfil:

Si ets...	Llegeix...	Conté
Jugador / usuari final	Visió General	Com instal·lar i jugar
Desenvolupador / professor	Arquitectura General	Com està fet per dins

Mapa de la documentació

Guia d'Usuari

- Visió General — què és el joc i com executar-lo
- Menú Principal — la pantalla d'inici
- Configuració de Partida — preparar una partida nova
- Tauler de Joc — la pantalla principal de joc
- Caselles Especials — què fa cada tipus de casella
- Inventari i Objectes — boles de neu, peixos i daus
- Guardar i Carregar — partides desades i autenticació
- Mode Debug — drecera per a desenvolupadors
- Idiomes i Localització — com canviar d'idioma

Guia Tècnica

- Arquitectura General — visió MVC global
- Estructura de Paquets — què conté cada paquet
- Paquet model.board · Paquet model.entity · Paquet model.item
- Paquet model.game · Paquet model.config · Paquet model.db
- Paquet controller · Paquet view
- Sistema de Sprites · Sistema de So · Sistema d'Idiomes · Sistema de Persistència

- [Diagrama de Classes](#) — UML general
- [Flux de Joc](#) — diagrames de seqüència
- [Mode Debug Tècnic](#) — implementació de Ctrl+Shift+D

Crèdits

TECNOLOGIA

- **Java 21 + JavaFX 21** (FXML + CSS)
- **SnakeYAML** per a localització i save-games
- **Oracle JDBC** per a la persistència (BBDD del curs)
- **AES-128** per encriptar contrasenyes i partides desades

Per als detalls tècnics complets consulta [Arquitectura General](#).

PART I - GUIA D'USUARI

Visió General

EN UNA FRASE

Joc del Pingu és un joc de taula digital per 1-4 jugadors locals on uns pingüins corren a arribar al final d'un camí de gel evitant ossos, forats i una foca enemiga.

Resum del joc

- **Format:** taulell snake (serp) de **50 caselles** (10 columnes × 5 files).
- **Jugadors:** 1 a 4 humans al mateix dispositiu, per torns.
- **Antagonista opcional:** una **foca CPU** (Seal) que també es mou i pot guanyar.
- **Objectiu:** ser el primer en arribar a la casella final.
- **Estil visual:** *pixel-art* a 17×19 píxels per pingüí, amb tintat per color de jugador.

Requeriments

Component	Versió mínima
Java	JDK 21 (mòduls de JavaFX inclosos a <code>module-info.java</code>)
JavaFX	21 (graphics, controls, fxml, media, swing)
SnakeYAML	qualsevol versió 1.x/2.x
Oracle JDBC	<code>ojdbc8</code> o superior (per BBDD del curs)
Sistema	Windows, macOS o Linux amb suport per JavaFX

CONNEXIÓ A BASE DE DADES

Sense la BBDD Oracle del curs no es poden desar partides ni veure estadístiques. La connexió està hardcodejada a `BBDD.java` amb credencials `DW2526_GR06_PINGU / ABDJFMV` al host `oracle.ilerna.com:1521/XEPDB2`.

Com iniciar el joc

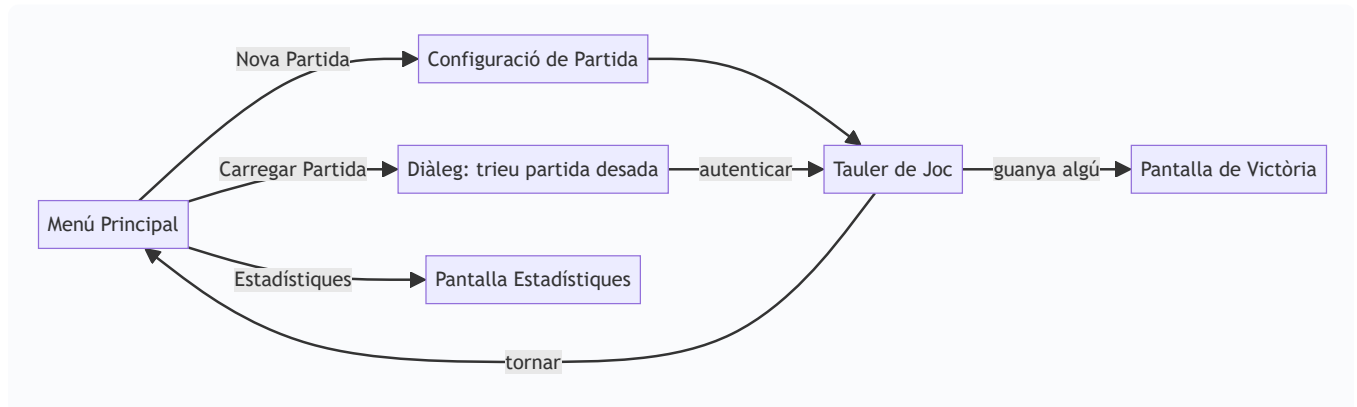
1. Obre el projecte a Eclipse (workspace `JocDelPingu`).
2. Comprova que `module-info.java` exporta els paquets necessaris.
3. Executa la classe principal `controller.main.MainMenu` (que estén `javafx.application.Application`).
4. S'obre la finestra a 900×720 px amb el **Menú Principal**.

```
// Punt d'entrada
public class MainMenu extends Application {
    public static void main(String[] args) {
        LangConfig.loadLang(); // carrega "en.yml" per defecte
        launch(args);
    }
}
```

Què passa quan arrenques

1. **Càrrega de fonts pixelades** (`pixel-game.regular.otf`, `pixel-game.extrude.otf`, `pixel-unicode-regular.ttf`).
2. **Càrrega del FXML del menú principal** (`mainMenu.fxml`) + l'estil CSS (`style.css`).
3. **Inici de la música de títol** (`main_screen_music.wav`, veure [Sistema de So](#)).
4. Es mostra el **Menú Principal** amb els botons traduïts a l'idioma per defecte (anglès).

Pantalles del joc



Cada bloc està documentat en una pàgina pròpia: [Menú Principal](#), [Configuració de Partida](#), [Tauler de Joc](#), [Guardar i Carregar](#).

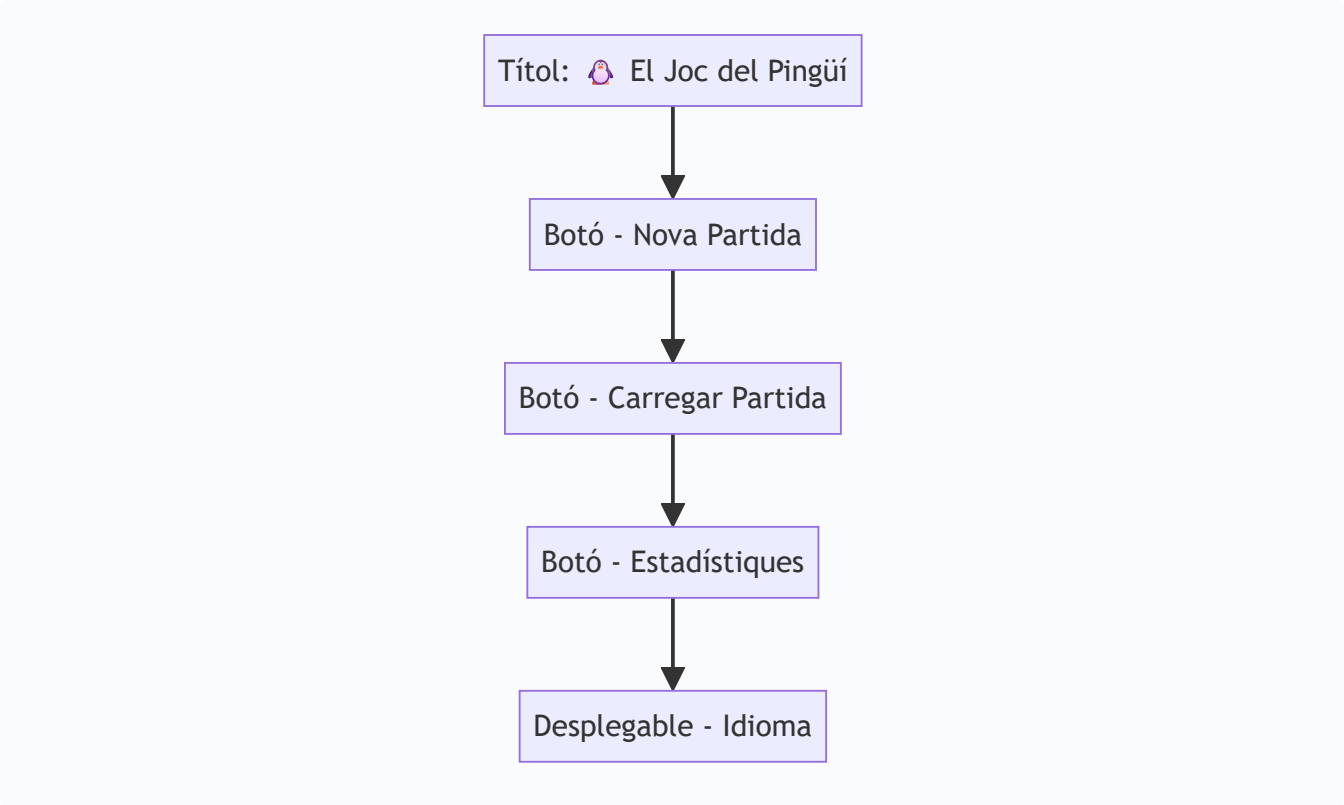
Aprèn més

- Quines caselles existeixen: [Caselles Especials](#)
- Quins objectes hi ha: [Inventari i Objectes](#)
- Com canviar d'idioma: [Idiomes i Localització](#)
- Si ets desenvolupador, comença per [Arquitectura General](#).

Menú Principal

És la primera pantalla que apareix en obrir el joc. Controlat per `MainMenuController` (veure [Paquet controller](#)) i descrit a `mainMenu.fxml` (veure [Paquet view](#)).

Aparença



Al fons hi ha la imatge `assets/sprites/backgrounds/1.png` carregada des de `MainMenuController.addBackgroundImage`. La música de fons és `main_screen_music.wav` (veure [Sistema de So](#)).

Botons

Botó	Mètode FXML	Què fa
Nova Partida	<code>#handleNewGame</code>	Carrega <code>playerSetup.fxml</code> → Configuració de Partida
Carregar Partida	<code>#handleLoadGame</code>	Mostra un <code>ChoiceDialog</code> amb els <code>GAME_ID</code> desats a Oracle
📊 Estadístiques	<code>#handleStats</code>	Carrega <code>playerStats.fxml</code> → Llista de jugadors registrats
🌐 Idioma (ComboBox)	listener intern	Crida <code>LangConfig.loadLang(code)</code> i refresca tots els textos

Selector d'idioma

És un `ComboBox<String>` (`language_combobox`) amb 11 entrades. Quan canvia la selecció:

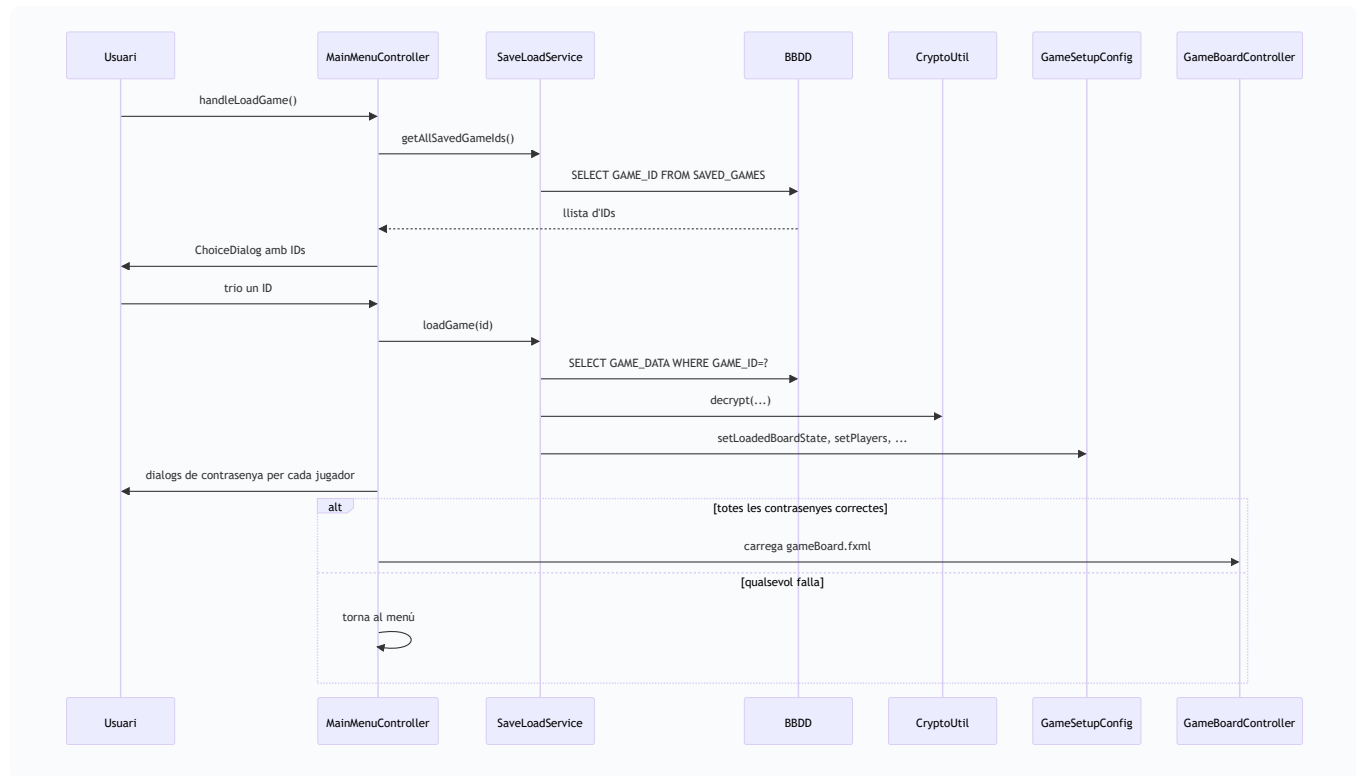
- Es busca el codi corresponent al nom mostrat (`LangConfig.getDisplayName`).

2. Es crida `LangConfig.loadLang(code)`.
3. Tots els *listeners* registrats amb `LangConfig.addLanguageChangeListener(...)` es disparen i refresquen els seus textos.

Veure [Idiomes i Localització](#) per la llista completa d'idiomes.

Carregar partida — flux

Quan cliques *Carregar Partida*:



DETALL IMPORTANT

Quan carregues una partida cal **autenticar tots els jugadors un per un** amb la seva contrasenya (veure `authenticateLoadedPlayers()`).

Cancel·lació segura

Si l'usuari cancel·la el diàleg d'autenticació al carregar una partida, el flag `GameSetupConfig.setLoadedGame(false)` es resetja per evitar que la pantalla següent es quedi en un estat a mig carregar.

Enllaços relacionats

- [Configuració de Partida](#) — pas següent en una nova partida
- [Guardar i Carregar](#) — detalls de la persistència
- [Sistema d'Idiomes](#) — detalls tècnics del selector

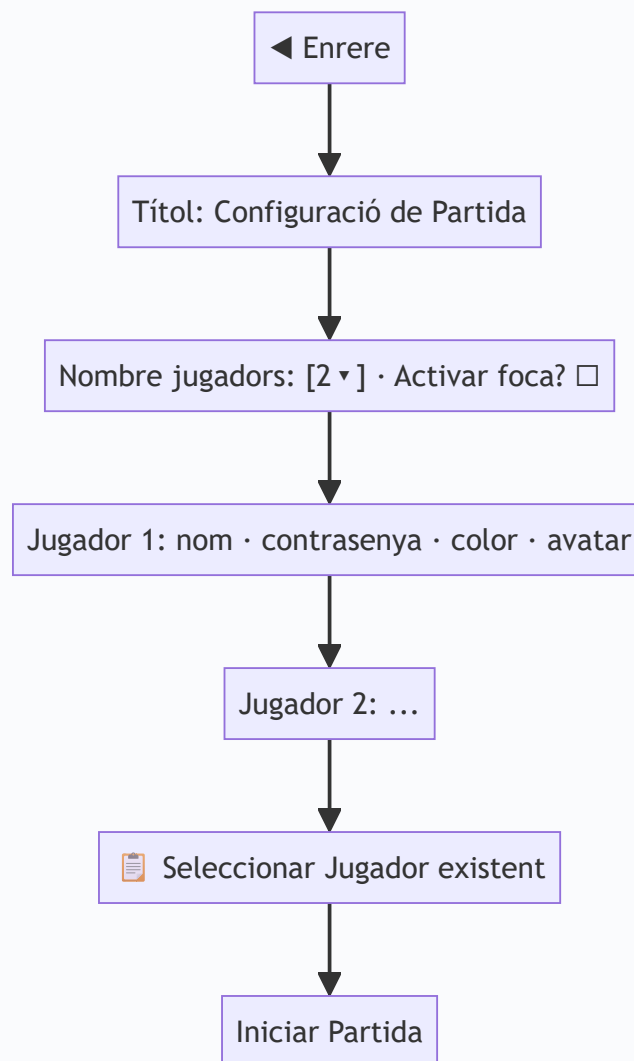
Configuració de Partida

Pantalla on es prepara una partida nova. Controlada per `PlayerSetupController` (Paquet controller) i descrita a `playerSetup.fxml` (Paquet view).

Què pots configurar

1. **Nombre de jugadors:** de **1 a 4** (`ComboBox<Integer>`).
2. **Activar foca?:** checkbox que activa la foca CPU com a rival comú.
3. **Per cada jugador:**
 - Nom (`TextField`)
 - Contrasenya (`PasswordField`)
 - Color (`ColorPicker`)
 - Avatar (botó *Tria una imatge d'avatar* → `FileChooser` per PNG/JPG/JPEG/GIF)

Estructura visual



Quan canvis el nombre de jugadors al desplegable es regeneren les fitxes (`updatePlayerFields`). Cada fitxa és una `VBox` amb classe CSS `setup-card` .

Seleccionar jugador existent

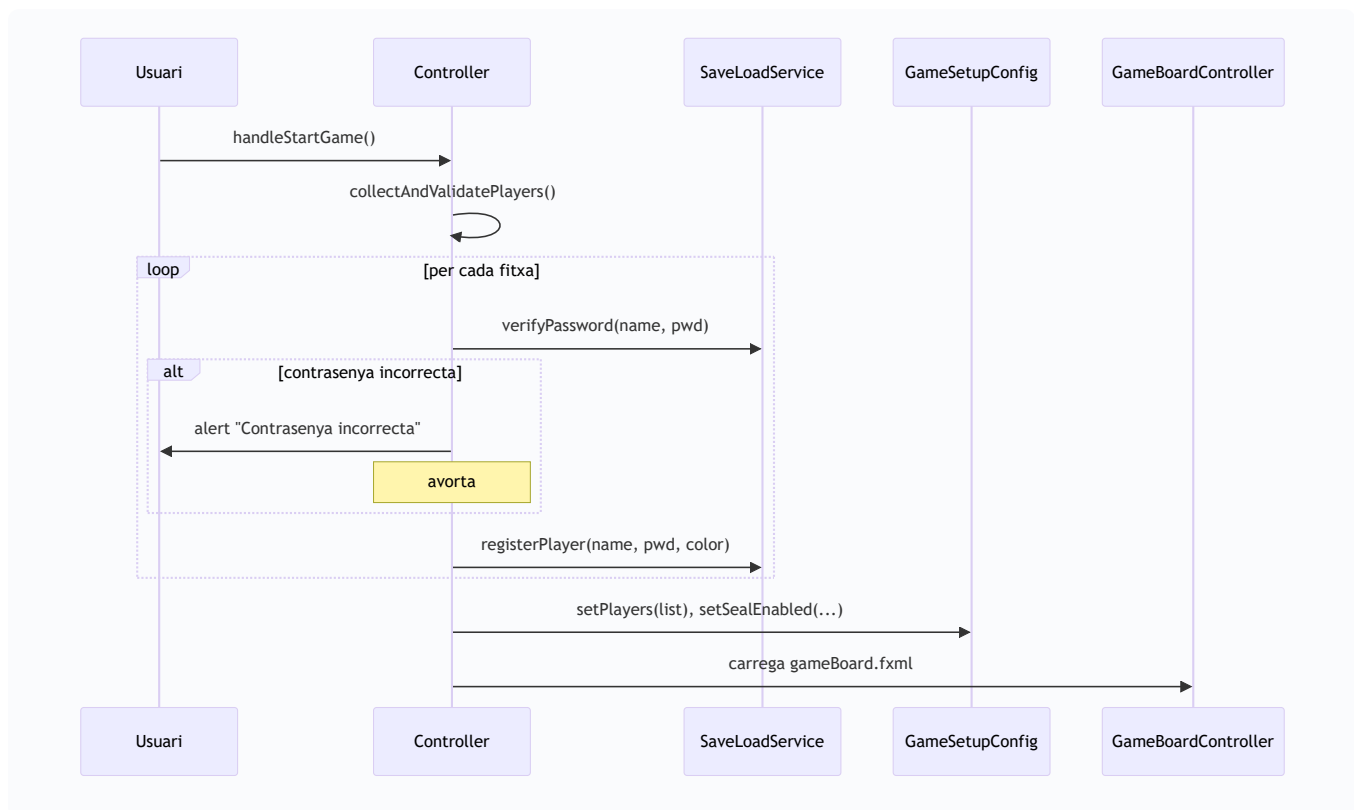
El botó  **Seleccionar Jugador** crida `handleSelectExistingPlayer()` . Mostra un `ChoiceDialog` amb els jugadors ja registrats a la taula `ENTITY` de la BBDD i omple la primera fitxa buida amb les dades del jugador triat (nom, color, contrasenya descriptada).

TOTS ELS ESPAIS PLENS

Si totes les fitxes ja tenen nom escrit i no en queda cap buida, es mostra un alert `ALERT_FULL_TITLE` (clau `alert.full.title`).

Iniciar partida

En fer clic a **Iniciar Partida** s'executa `handleStartGame()` :



VALIDACIÓ DE CONTRASENYES

`SaveLoadService.verifyPassword` busca a la taula `ENTITY` . Si el jugador **no existeix encara** retorna `true` (es considera un alta nova). Si **existeix**, compara la contrasenya escrita amb el valor descriptat de la BBDD.

Camps tècnics rellevants

Camp	Tipus	Significat
numPlayersCombo	ComboBox<Integer>	1, 2, 3 o 4
sealCheckBox	CheckBox	Activa la foca CPU
playerInputs	List<PlayerInput>	Una fitxa per jugador (classe interna)
color	String hex (6 dígits)	Es deriva del <code>ColorPicker</code> retallant el prefix <code>0x</code> i l'alfa
avatar	String URI	URI del fitxer triat (<code>file:///...</code>)

El que queda guardat a `GameSetupConfig`

Veure `GameSetupConfig`:

```
GameSetupConfig.setPlayers(players);           // List<Player>
GameSetupConfig.setSealEnabled(sealCheckBox.isSelected());
GameSetupConfig.setLoadedGame(false);          // és partida nova, no carregada
```

Enllaços relacionats

- [Menú Principal](#) — pantalla anterior
- [Tauler de Joc](#) — pantalla següent
- [Guardar i Carregar](#) — com es registren els jugadors a BBDD

Tauler de Joc

És la pantalla principal del joc, controlada per `GameBoardController` (1780+ línies, vegeu [Paquet controller](#)) i descrita a `gameBoard.fxml`.

Disseny snake (serp)

El tauler és una graella de **10 columnes × 5 files = 50 caselles** (`Board.widthBoard=10`, `Board.heightBoard=5`, `Board.MAX_SQUARES=50`). Les caselles s'ordenen en zig-zag (com una serp):

```
[ 0][ 1][ 2][ 3][ 4][ 5][ 6][ 7][ 8][ 9] ← fila 0 (esquerra→dreta)
[19][18][17][16][15][14][13][12][11][10] ← fila 1 (dreta→esquerra)
[20][21][22][23][24][25][26][27][28][29] ← fila 2
[39][38][37][36][35][34][33][32][31][30] ← fila 3
[40][41][42][43][44][45][46][47][48][49] ← fila 4
```

DIRECCIÓ DE MIRADA

En files parells (0, 2, 4) els pingüins miren a la **dreta**; en files senars miren a l'**esquerra**. Això ho gestiona `renderPlayersOnCell()` amb la fórmula `rowFacesRight = (row % 2 == 0)`.

Layout de pantalla (BorderPane)

TOP: Hotbar dinàmica - avatar, inventari, títol

CENTER: GridPane 10x5 amb caselles

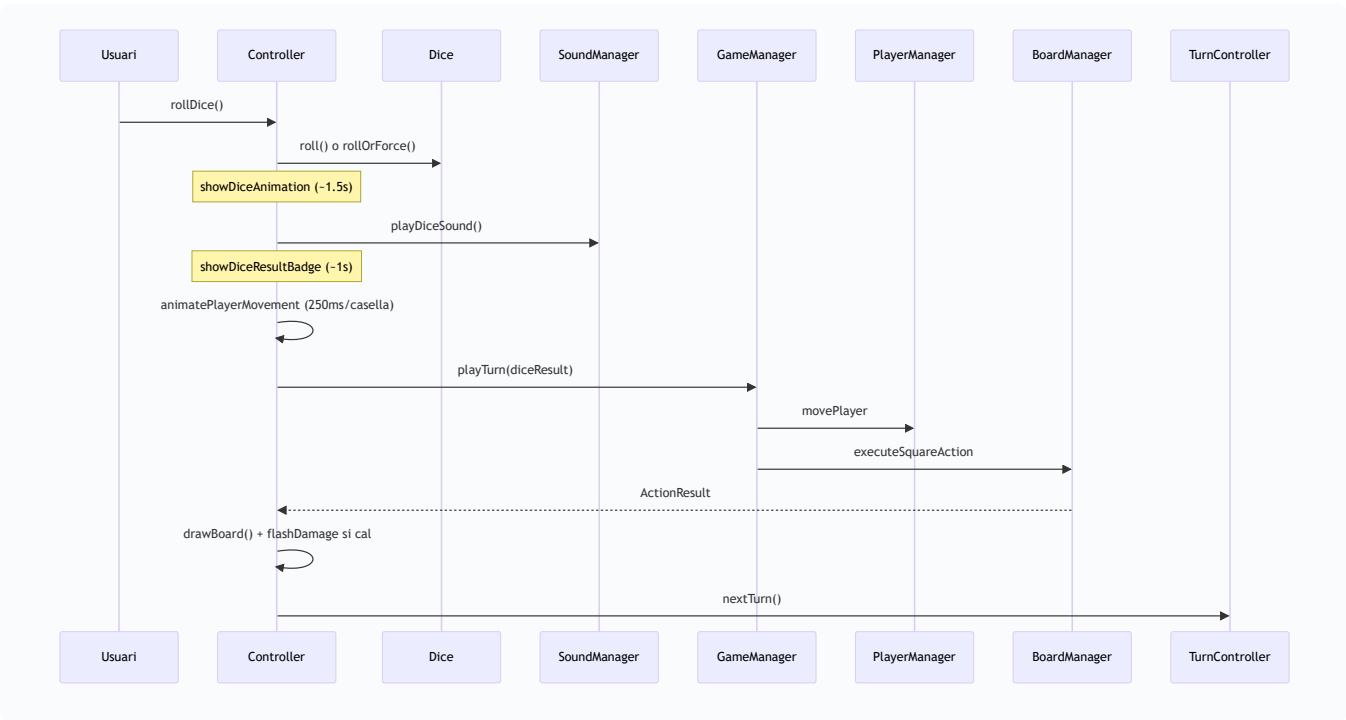
BOTTOM: Action Bar - botons

- **TOP:** una `HBox` construïda dinàmicament a `createHotbar(player)` amb retrat + nom + slots d'inventari + títol del joc.
- **CENTER:** `boardContainer > GridPane#grid` redibuixat a `drawBoard()` cada vegada que es modifica l'estat o canvia la mida de finestra.
- **BOTTOM:** `FlowPane.action-bar` amb els botons d'acció.

Botons de l'Action Bar

Botó	Mètode	Què fa
Roll Dice	#rollDice	Tira el dau normal (1-6)
Fast Dice	#rollFastDice	Tira un dau ràpid (5-10) si en tens
Slow Dice	#rollSlowDice	Tira un dau lent (1-3) si en tens
Throw Snowball	#throwSnowball	Llança una bola de neu a un altre jugador
Save Game	#saveGame	Guarda la partida (demana nom)
History	#showEventHistory	Mostra el log d'esdeveniments
Back	#handleBack	Torna a la pantalla anterior
Menu	#handleReturnToMenu	Torna al menú principal

Tirada de dau



Veure [Flux de Joc](#) per als diagrames complets.

Gir de torn

Després que el jugador actual completi el seu moviment es crida `endTurn()` que crida `turnController.nextTurn()`. Aquest:

1. Avança l'índex modulant per la mida de la llista.
2. Si el següent jugador té `shouldSkipNextTurn() == true` (efecte d'una casella que el va sancionar), li reseteja el flag i salta una posició més.

3. Si la foca està activada i és el torn 0, primer juga la foca (`playSealTurnAnimated`) i després passa al primer jugador humà.

PINGÜÍ CONGELAT

Si un jugador ha caigut a un forat de gel a la jugada anterior, el seu sprite està congelat (`isFrozen==true`). En la seva pròxima tirada de dau el flag es treu i el sprite torna a la normalitat. Veure [Sistema de Sprites](#).

HUD i hotbar

A la part superior es mostra una **hotbar** amb:

- **Retrat** del jugador actual (avatar personalitzat o sprite tintat amb `Lighting` segons el seu color).
- **Nom** del jugador actual.
- **Inventory slots**: només es mostren els objectes que té (snowball, fish, fastdice, slowdice) amb la quantitat com a badge.
- **Títol "Pingu's Game"** centrat com a decoració.

A la dreta (panell `rightPanel`, opcional/ocult per defecte) es pot mostrar informació detallada del jugador, comptadors d'inventari i estat de la foca.

Estat de la foca

Si `sealEnabled == true`, dins el panell dret apareix un `VBox#sealStatusBox` amb:

-  Posició actual al tauler
-  Estat (activa) o  *Eating fish (X turns left)* si està bloquejada per un peix.

Veure [Seal](#) per detalls del comportament.

Esdeveniments del joc

Cada acció (tirada, casella, atac, etc.) genera un `String` que:

- Va al log accessible via  **History** (`showEventHistory()`).
- Es desa al `Player.eventHistory` (màxim 50 entrades) perquè sobrevisqui al guardat.

Enllaços relacionats

- [Caselles Especials](#) — efectes per tipus de casella
- [Inventari i Objectes](#) — com s'utilitzen els objectes
- [Mode Debug](#) — drecera per als desenvolupadors
- [Sistema de Sprites](#) — com es dibuixen els pingüins
- [Flux de Joc](#) — diagrames de seqüència detallats

Caselles Especials

El tauler té **8 tipus de casella** definits a l'enum `SquareType` (Paquet `model.board`). Quan es genera un tauler nou, el **70%** són casellas normals (`Board.NORMAL_SQUARE_PERCENTAGE = 70`) i el 30% restant es reparteix aleatòriament entre els 5 tipus especials. La casella **0** sempre és `START` i la **49** sempre és `END`.

Taula resum

Tipus	Classe	Efecte	Restablible
NORMAL	S_Normal	Cap efecte — log neutre	—
START	S_Start	Punt de sortida (casella 0)	—
END	S_End	Final — victòria	—
BEAR	S_Bear	Si tens peix: en gastes un i et quedes. Sinó: tornes a 0.	—
ICE_HOLE	S_IceHole	Et tira enrere al forat anterior (o a 0). Congela el sprite.	—
SLED	S_Sled	Et tira endavant fins al següent trineu.	—
EVENT	S_Event	Esdeveniment aleatori (veure <code>EventManager</code>)	—
BROKEN_FLOOR	S_BrokenFloor	Depèn de quants objectes portis	Es converteix en <code>ICE_HOLE</code> si caus

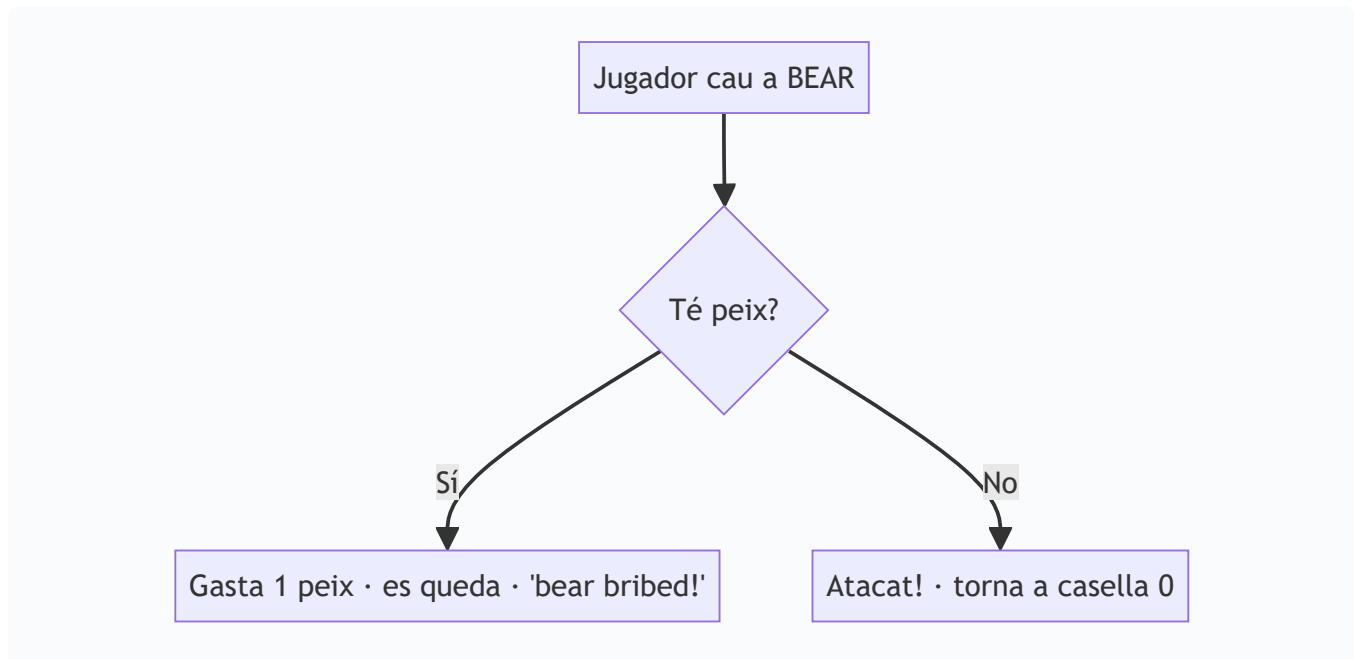
NORMAL

Casella de farciment. Només produeix un missatge de log: *"Jugador X landed on a normal square"*. Sense efectes sobre l'estat.

START i END

- **START** (índex 0): tots els jugadors hi comencen. Els efectes que diuen "torna a l'inici" hi porten el jugador.
- **END** (índex 49): el primer jugador que hi arribi guanya. La detecció final es fa a `PlayerManager.movePlayer()` quan el resultat de la suma supera `MAX_SQUARES - 1`.

BEAR (🐻 Ós polar)



COMPENSACIÓ

Els peixos són l'únic recurs que evita perdre tot l'avenç a una casella BEAR.

ICE_HOLE (🧊 Forat de gel)

Llista ordenada `Board.IceHole_Array`. Quan caus a un forat, ets enviat **al forat anterior de la llista**. Si caus al primer forat, vas a la casella 0.

SPRITE CONGELAT

A més de moure't, et marca com a `frozen=true` perquè es mostri el sprite gelat (`ice_player_*.png`). Es desbloqueja quan tornes a tirar el dau.

SLED (🛷 Trineu)

És l'invers del forat: et tira al **següent trineu** de la llista `Board.Sled_Array`. Si caus al darrer trineu, no et mou.

EVENT (? Esdeveniment aleatori)

Es resol a través de `EventManager.triggerEvent()` amb les probabilitats següents:

Probabilitat	Esdeveniment	Efecte
30%	GET_SLOW_DICE	Reps un dau lent (1-3) si l'inventari no està ple
20%	GET_SNOWBALLS	Reps 1-3 boles de neu
15%	GET_FISH	Reps un peix
12%	LOSE_TURN	Pateixes pell de plàtan: perds el següent torn
10%	LOSE_ITEM	Una ràfega de vent et fa caure un objecte aleatori
8%	GET_FAST_DICE	Reps un dau ràpid (5-10) — esdeveniment poc freqüent
5%	SNOWMOBILE	Vehicle de neu: avança fins al trineu següent

SENSE DUPLICACIÓ

`S_Event.action()` és buit a propòsit; la resolució autèntica passa al `BoardManager.handleEvent()` que crida l' `EventManager` .

BROKEN_FLOOR (💔 Terra trencat)

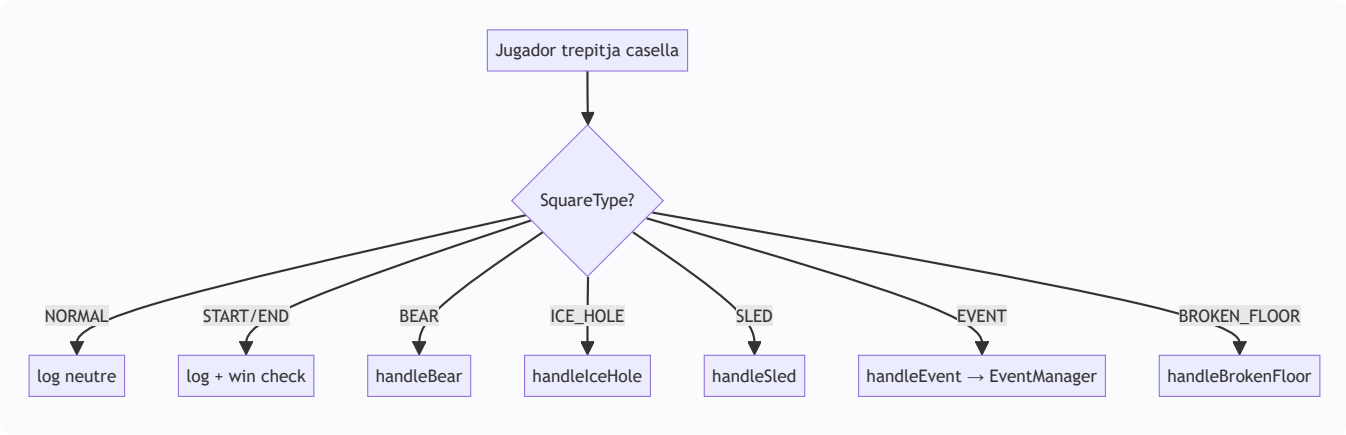
L'efecte depèn de quants **objectes totals** portis (`Inventory.getTotalItemCount()`):

Items totals	Resultat
> 5	Caus i tornes a 0. La casella es transforma en ICE_HOLE permanentment (<code>Board.convertBrokenFloorToIceHole</code>).
1 a 5	50% → perds un objecte aleatori. 50% → perds el següent torn.
0	Sense problemes — passes sense penalització.

COMPROMÍS ESTRATÈGIC

Acumular molts objectes ajuda contra BEAR i EVENT, però et fa vulnerable a BROKEN_FLOOR. Pensa-t'ho bé.

Diagrama de resolució (BoardManager)



Enllaços relacionats

- [Paquet model.board](#) — implementació de cada classe `S_*`
- [Inventari i Objectes](#) — com els objectes afecten les decisions
- [Flux de Joc](#) — seqüència completa d'un torn

Inventari i Objectes

Cada Player té un **Inventory** (Paquet `model.item`) que emmagatzema quatre tipus d'objectes apilables. La foca CPU no té inventari.

Objectes disponibles

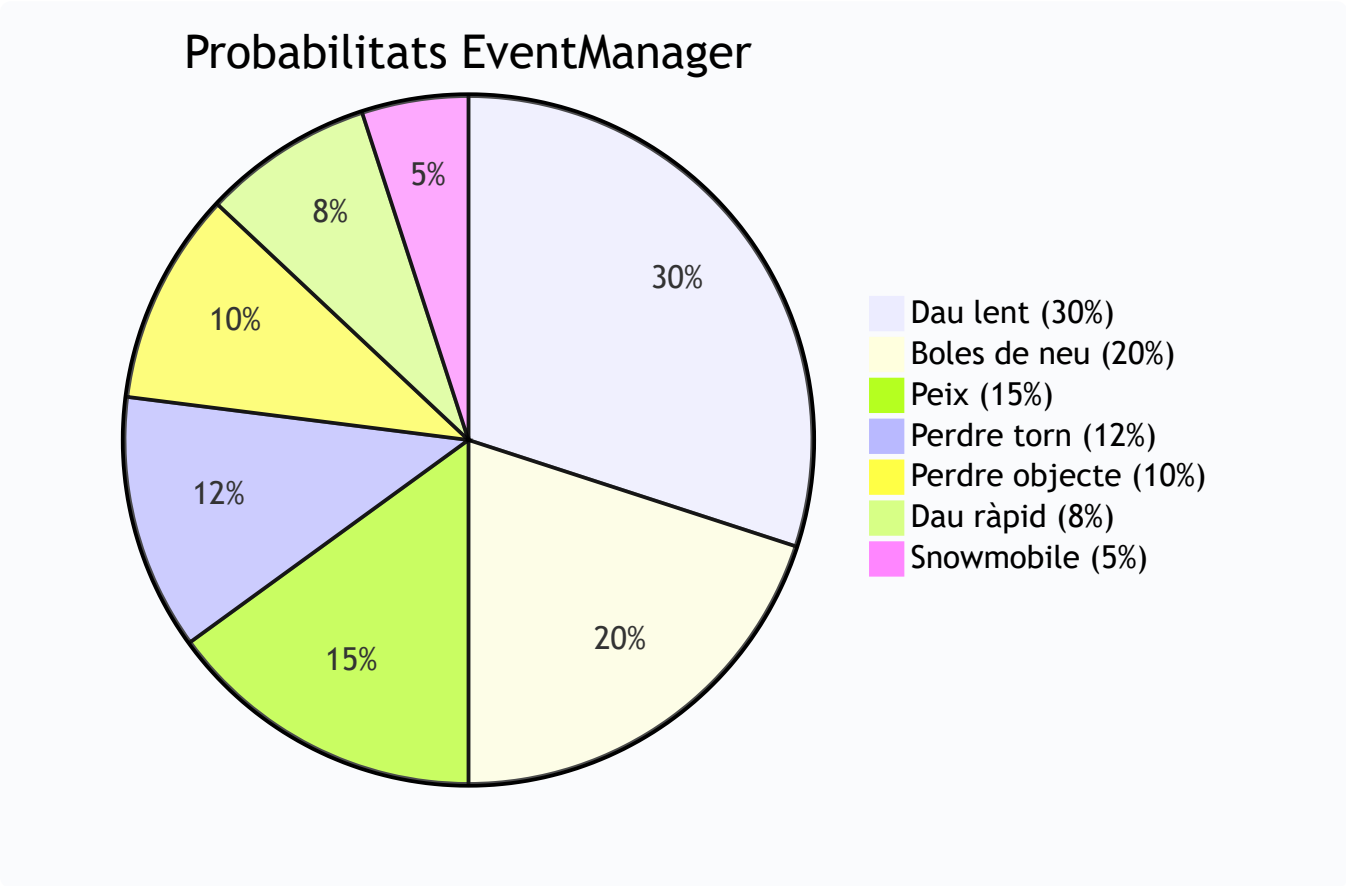
Objecte	Tipus (ObjectType)	Màxim	Per què serveix
 Bola de neu	SNOWBALL	6 (MAX_SNOWBALLS)	Llançar a un altre jugador per fer-lo retrocedir 1-3 caselles
 Peix	FISH	2 (MAX_FISH)	Suborna l'ós a BEAR o la foca
  Dau ràpid	FASTDICE	(compartit)	Tira un dau de 5-10 (avenç gran)
 Dau lent	SLOWDICE	(compartit)	Tira un dau d'1-3 (avenç curt segur)

CAP TOTAL DE DAUS

Els daus comparteixen una **capacitat conjunta** de 3 (MAX_DICE = 3). Pot ser 3 ràpids, 3 lents o qualsevol combinació que sumi 3 — però mai més.

Com s'obtenen?

Tots els objectes provenen d'una casella **EVENT** (veure [Caselles Especials](#)). Les probabilitats estan a [EventManager](#):



Boles de neu (Snowball)

Es llancen amb el botó  **Throw Snowball** (Tauler de Joc). Hi ha dos modes:

1. **Llançament dirigit** (`PlayerManager.throwSnowball`): tries un objectiu, gastes 1 bola, l'objectiu retrocedeix **1-3 caselles** (aleatori) i el seu sprite parpelleja com a *damaged*.
2. **Snowball War** (`PlayerManager.snowballWar`): es dispara automàticament quan dos jugadors caiguin a la **mateixa casella**. Ambdós gasten **totes** les seves boles, qui en tenia més guanya i l'altre retrocedeix per la diferència.

EMPAT

Si tots dos en tenen la mateixa quantitat (incloent 0), és empat i ningú retrocedeix.

Peixos (Fish)

Tenen dues utilitats defensives:

- A casella **BEAR**: subornes l'ós i et quedes (1 peix consumit).
- A casella ocupada per la **foca**: la bloqueges **2 torns** (`Seal.bribeSeal`).

RECURS ESCÀS

Només pots tenir **2 peixos a la motxilla** (`MAX_FISH = 2`). Si reps un peix amb la motxilla plena, no s'afegeix.

Daus especials

Dau	Rang	Mètode <code>Dice.roll()</code>
Normal (<code>DICE</code>)	1-6	<code>roll()</code> retorna entre <code>minValue=1</code> i <code>maxValue=6</code>
Ràpid (<code>FASTDICE</code>)	5-10	<code>FASTDICE_MIN_VALUE..FASTDICE_MAX_VALUE</code>
Lent (<code>SLOWDICE</code>)	1-3	<code>SLOWDICE_MIN_VALUE..SLOWDICE_MAX_VALUE</code>

L'ús d'un dau especial **el consumeix** (`useObject(FASTDICE, 1)`). Si no en tens, el botó queda deshabilitat:

```
rollFastDiceButton.setDisable(inv.getFastdiceQuantity() <= 0);
rollSlowDiceButton.setDisable(inv.getSlowdiceQuantity() <= 0);
throwSnowballButton.setDisable(inv.getSnowballQuantity() <= 0);
```

Mètodes clau de `Inventory`

Mètode	Què fa
<code>addSnowballs(n)</code>	Afegeix fins a <code>MAX_SNOWBALLS</code> , retorna quantes ha afegit
<code>addFish()</code>	Afegeix 1 si <code><MAX_FISH</code> , retorna booleà
<code>addDice(type)</code>	Afegeix 1 dau del tipus si <code>diceQuantity<MAX_DICE</code>
<code>useObject(type, q)</code>	Resta del comptador; retorna fals si no n'hi havia prou
<code>removeRandomItem()</code>	Elimina 1 objecte aleatori; retorna el tipus eliminat o <code>null</code>
<code>removeHalf()</code>	Divideix per 2 totes les quantitats (efecte de la foca passant)
<code>getTotalItemCount()</code>	Suma de tots (no inclou <code>DICE</code> genèric, sí inclou ràpid+lent)

Pèrdua d'objectes — quan i com

- **Esdeveniment `LOSE_ITEM` (10%)**: pèrdua aleatòria d'1 objecte.
- **`BROKEN_FLOOR` amb 1-5 objectes (50%)**: pèrdua aleatòria d'1 objecte.
- **`Sea1.passThrough()`**: la foca travessa la teva casella → perds la **meitat** de tots els objectes (`Inventory.removeHalf()`).

Diagrama de classes



Syntax error in text
mermaid version 10.9.6

Enllaços relacionats

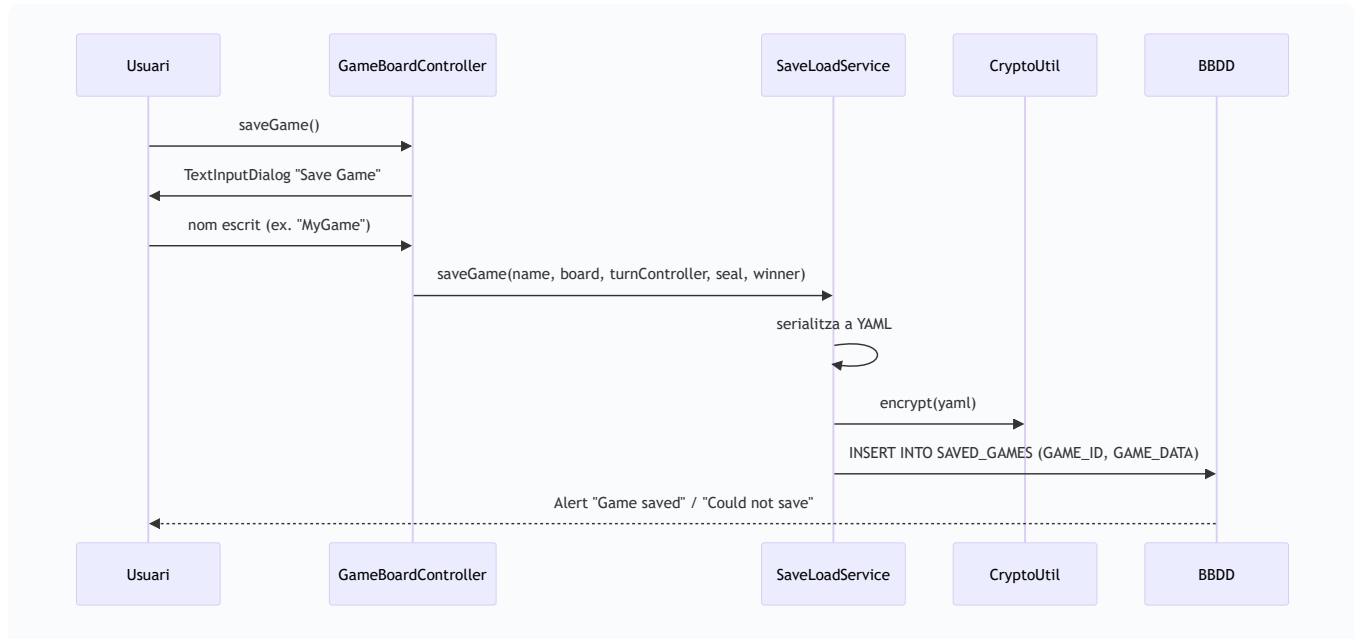
- Caselles Especials — d'on surten els objectes
- Paquet `model.item` — detalls tècnics
- Mode Debug — drecera per modificar inventaris durant proves

Guardar i Carregar

El joc permet **desar la partida actual a Oracle** i recuperar-la més tard. Tot ho gestiona `SaveLoadService` (Paquet `model.game`) amb encriptació AES (veure Sistema de Persistència).

Guardar partida

Des del Tauler de Joc, clica  **Save Game**. Apareix un `TextInputDialog`:



Què es desa exactament

L'estructura YAML que es genera (i s'encripta abans de pujar-la a la BBDD):

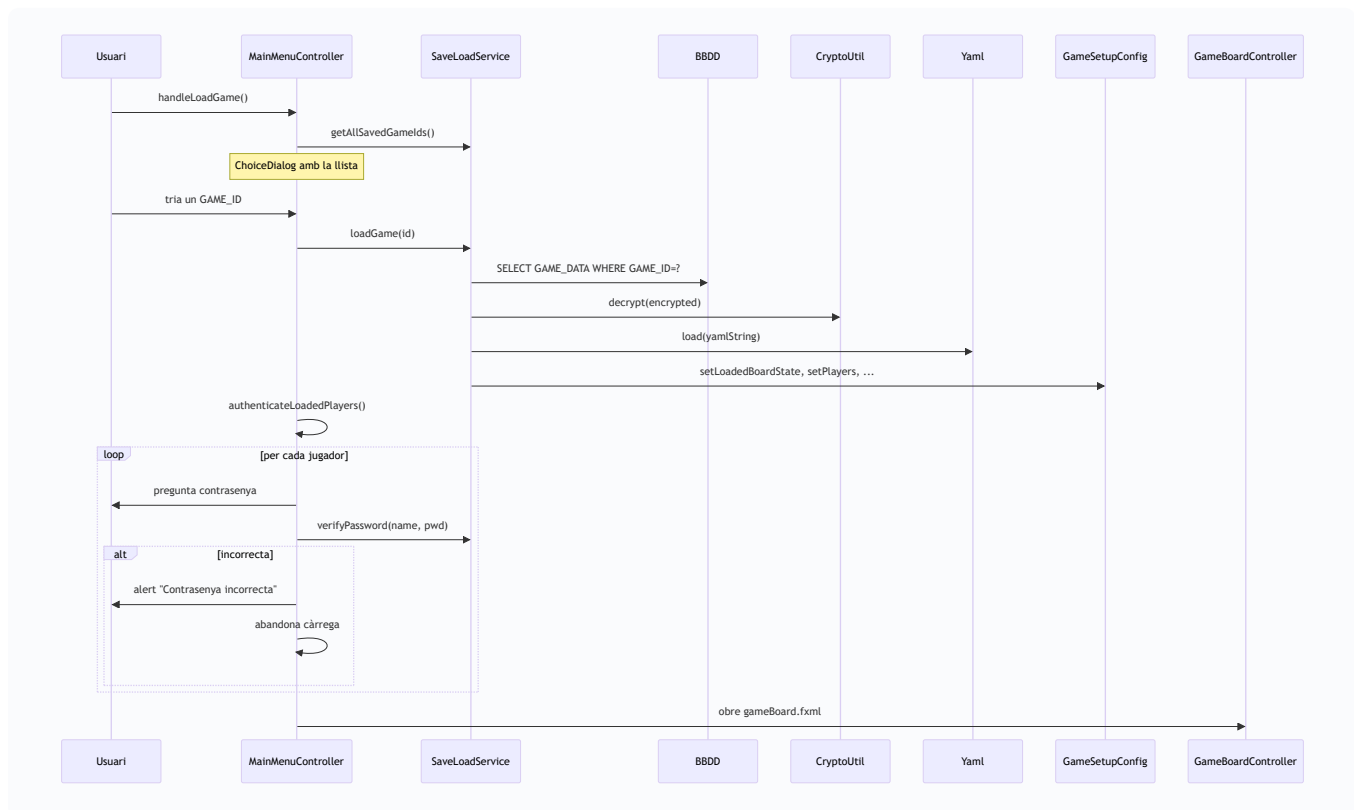
```
board: [START, NORMAL, ICE_HOLE, NORMAL, ...] # tipus de cada casella
currentTurn: 2
players:
  - name: Pingu
    color: FF0000
    password: hunter2
    id: 12345
    square: 17
    skipNextTurn: false
    inventory: { snowballs: 3, fish: 1, fastdice: 0, slowdice: 1 }
    eventHistory: [ "🐼 5 → ...", ... ] # màxim 50 entrades
seal: # només si la foca està activada
  square: 22
  blockedTurns: 0
```

CLOB I PREPAREDSTATEMENT

El YAML encriptat sovint supera els 4000 caràcters, per això s'insereix amb un `PreparedStatement` cap a un `CLOB (GAME_DATA)`. Una concatenació amb cometes simples falla amb ORA-01704.

Carregar partida

Des del Menú Principal clica **Carregar Partida**:



Autenticació de jugadors

PER QUÈ AUTENTICAR?

Així evitem que un altre usuari obri la partida d'algú altre sense permís. Cada jugador ha de saber la seva contrasenya.

A `MainMenuController.authenticateLoadedPlayers()`:

1. Obté els jugadors carregats de `GameSetupConfig.getPlayers()`.
2. Per cada jugador mostra un `Dialog<String>` amb un `PasswordField`.
3. Crida `SaveLoadService.verifyPassword(name, input)`:
 - Recupera la contrasenya **encriptada** de la taula `ENTITY` (clau primària `PLAYERNAME`).
 - Si no hi ha contrasenya guardada → s'accepta només si l'entrada és buida.
 - Sinó → `CryptoUtil.decrypt(stored).equals(input)`.
4. Si una contrasenya falla o l'usuari cancel·la, es torna al menú i es reseteja `setLoadedGame(false)`.

Registre automàtic de jugadors

Cada vegada que es comença una partida nova, `SaveLoadService.registerPlayer(name, password, color)` es crida per cada jugador. Utilitza un `MERGE SQL`:


```
MERGE INTO ENTITY e
USING (SELECT 'Pingu' AS pname FROM DUAL) src
ON (e.PLAYERNAME = src.pname AND e.ENTITYTYPE = 'PLAYER')
WHEN MATCHED THEN UPDATE SET PLAYERPASSWORD=..., COLOUR=...
WHEN NOT MATCHED THEN INSERT (...) VALUES (...)
```

Així el jugador s'**actualitza** si ja existeix o s'**insereix** si és nou. La contrasenya es desa encriptada via `CryptoUtil.encrypt()`.

Estadístiques

`SaveLoadService.recordGameResult()` es crida quan una partida acaba (`handleWin` o victòria de foca). Actualitza dues columnes amb `NVL` (per evitar el problema `NULL+1=NULL` d'Oracle):

Columna	Quan s'incrementa
<code>ENTITY.GAMES_PLAYED</code>	+1 per cada jugador humà
<code>ENTITY.GAMES_WON</code>	+1 només per al guanyador (si n'hi ha)

Les estadístiques es consulten des de **Estadístiques** via `getPlayerStats()`.

Taules Oracle implicades

Taula	Columnes rellevants
<code>ENTITY</code>	<code>ENTITYID</code> , <code>ENTITYTYPE='PLAYER'</code> , <code>PLAYERNAME</code> , <code>PLAYERPASSWORD (encrypt)</code> , <code>COLOUR</code> , <code>GAMES_PLAYED</code> , <code>GAMES_WON</code>
<code>SAVED_GAMES</code>	<code>GAME_ID</code> , <code>GAME_DATA (CLOB encrypted)</code>
<code>GAME</code>	<code>GAMEID</code> , <code>GAMESTATE</code> , <code>GAMEDATE</code> , <code>BOARDID</code> (insertat a <code>recordGameResult</code>)
<code>BOARD</code>	<code>BOARDID</code> (PK referenciada per <code>GAME.BOARDID</code>)

TRACTA DE SQL INJECTION

Algunes consultes concatenen strings amb `.replace("'", "''")` com a salvaguarda mínima. Per al projecte didàctic ho deixen així, però en producció caldria fer servir `PreparedStatement` en tot.

Enllaços relacionats

- [Sistema de Persistència](#) — detalls tècnics i AES
- [Paquet model.game](#) — `SaveLoadService` complet
- [Paquet model.db](#) — capa d'accés a Oracle (BBDD)

Mode Debug

FUNCIÓ OCULTA

Aquesta funcionalitat és per a desenvolupadors i testers. **No està anunciada a la UI**, però és perfectament funcional.

Activació

Quan estiguis al **Tauler de Joc**, prem la combinació:

Ctrl + Shift + D

Apareix un panell groc/negre a la part superior amb:

```
⚡ DEBUG ON   Force next roll: [____] [Set] [Clear]   Drag any 🐧 to teleport.  
Inventory of: [jugador▼]   🎲 - [+/-]   🐟 - [+/-]   🎲🌟 - [+/-]   🎲 - [+/-]
```

Pots desactivar-lo prement la mateixa combinació una segona vegada.

Què pots fer

1. Forçar un valor de dau

Escriu un nombre entre **1 i 50** al camp de text i clica **Set**. La pròxima tirada (de qualsevol dau: normal, ràpid o lent) usarà aquest valor en lloc d'un de aleatori. Després es consumeix automàticament.

Útil per provar caselles específiques sense haver de tornar a llençar el dau repetidament.

2. Teletransportar un pingüí

Amb el mode debug actiu, **arrossega qualsevol pingüí amb el ratolí** fins a la casella que vulguis. En soltar-lo, el jugador queda a la nova posició.

```
Mantenir clic sobre pingüí → Arrossegar pel tauler → Soltar a una casella → Pingüí mogut · log: ⚡ DEBUG: teleported
```

El cursor canvia a `OPEN_HAND` quan es passa per sobre i a `CLOSED_HAND` mentre s'està arrossegant.

3. Modificar inventaris

Al combobox **Inventory of:** tria el jugador. A continuació, pots:

- Clicar **+** per afegir un objecte (respectant els màxims: `MAX_SNOWBALLS=6`, `MAX_FISH=2`, `MAX_DICE=3` total).
- Clicar **-** per treure'n un (si en queda almenys 1).

Els canvis s'apliquen immediatament i el HUD es refresca.

Per a què serveix?

Cas d'ús	Com fer-ho
Provar la casella d'esdeveniment	Forçar tirada perquè caigui exactament sobre una EVENT
Provar BROKEN_FLOOR amb molts items	Afegir 6+ items i teletransportar al pingüí allà
Reproduir una partida bug	Teletransportar i ajustar inventaris fins arribar a la situació problemàtica
Demostrar la victòria	Forçar un dau alt o teletransportar al pingüí a la casella 48

Limitacions

- El mode **no es desa**: si carregues una partida, t'haurà de reactivar-lo manualment.
- No funciona durant les animacions (haur d'esperar que acabin).
- No pots teletransportar la **foca** (només jugadors humans).

Enllaços relacionats

- [Mode Debug Tècnic](#) — com està implementat per dins (event filter + drag handlers)
- [Tauler de Joc](#) — la pantalla on s'activa

Idiomes i Localització

El joc està totalment localitzat. Pots canviar d'idioma des del [Menú Principal](#) amb el desplegable  **Idioma**.

Idiomes suportats

Codi	Idioma	Etiqueta al menú
en	Anglès (per defecte)	 English
es	Espanyol	 Español
ca	Català	 Català
fr	Francès	 Français
pt	Portuguès	 Português
ro	Romanès	 Română
ar	Àrab	 العربية
ru	Rus	 Русский
uk	Ucraïnès	 Українська
ff	Pulaar	 Pulaar
jp	Japonès (transliterat)	 Japanese

PER QUÈ PULAAR I JAPONÈS EN ALFABET LLATÍ?

Les fonts *pixel-art* del joc (`pixel-game.regular.otf`) no contenen els glifs Adlam (U+1E900+) ni CJK, per això aquests idiomes es renderitzen amb caràcters llatins.

Com canviar d'idioma

1. A la pantalla del [Menú Principal](#), fes clic al desplegable groc al final dels botons.
2. Tria un dels onze idiomes.
3. **Tots els textos** es refresquen a l'instant — botons, títol de finestra, etiquetes, diàlegs.

PERSISTÈNCIA

El canvi és per a la sessió actual; en obrir el joc una altra vegada torna a l'idioma per defecte (`en`).

Fitxers de traducció

Les traduccions són **fitxers YAML** a `src/assets/lang/`:

```
ca.yml  en.yml  es.yml  fr.yml  pt.yml  ro.yml
ar.yml  ru.yml  uk.yml  ff.yml  jp.yml
```

Estructura (extracte de `ca.yml`):

```
text.game.title: "El Joc del Pingüí"
menu.button.newgame: "Nova Partida"
menu.button.loadgame: "Carregar Partida"
menu.button.stats: "📊 Estadístiques"
alert.wrongpassword.title: "Contrasenya incorrecta"
alert.wrongpassword.message: "La contrasenya del jugador '%s' és incorrecta..."
```

Les claus s'identifiquen amb l'enum `Lang` ([Paquet model.config](#)) que mapa cada constant Java al seu key YAML:

```
LangConfig.getLang(Lang.MENU_BUTTON_NEWGAME); // → "Nova Partida" (en català)
```

Refresc dinàmic

Cada pantalla registra un *listener* a l'inicialització:

```
LangConfig.addLanguageChangeListener(this::refreshTexts);
```

Quan algú canvia d'idioma, `LangConfig.loadLang(code)` notifica tots els *listeners* i cadascun crida el seu mètode `refreshTexts()` per redibuixar etiquetes. Veure [Sistema d'Idiomes](#) per a la implementació detallada.

Què està traduït i què no

Traduït	No traduït
Botons del menú	Missatges del log de joc (anglès)
Títol de la finestra	Logs de la consola
Camps de configuració	Noms dels jugadors
Diàlegs d'alerta	—
Pantalla d'estadístiques	—

LOG DE JOC EN ANGLÈS

Els missatges com *"Pingu rolled 4 (Normal die)"* o *"Bear attacked!"* només existeixen en anglès al codi (`formatActionMessage` de `GameBoardController`).

Enllaços relacionats

- [Sistema d'Idiomes](#) — detalls tècnics
- [Paquet model.config](#) — `Lang` i `LangConfig`

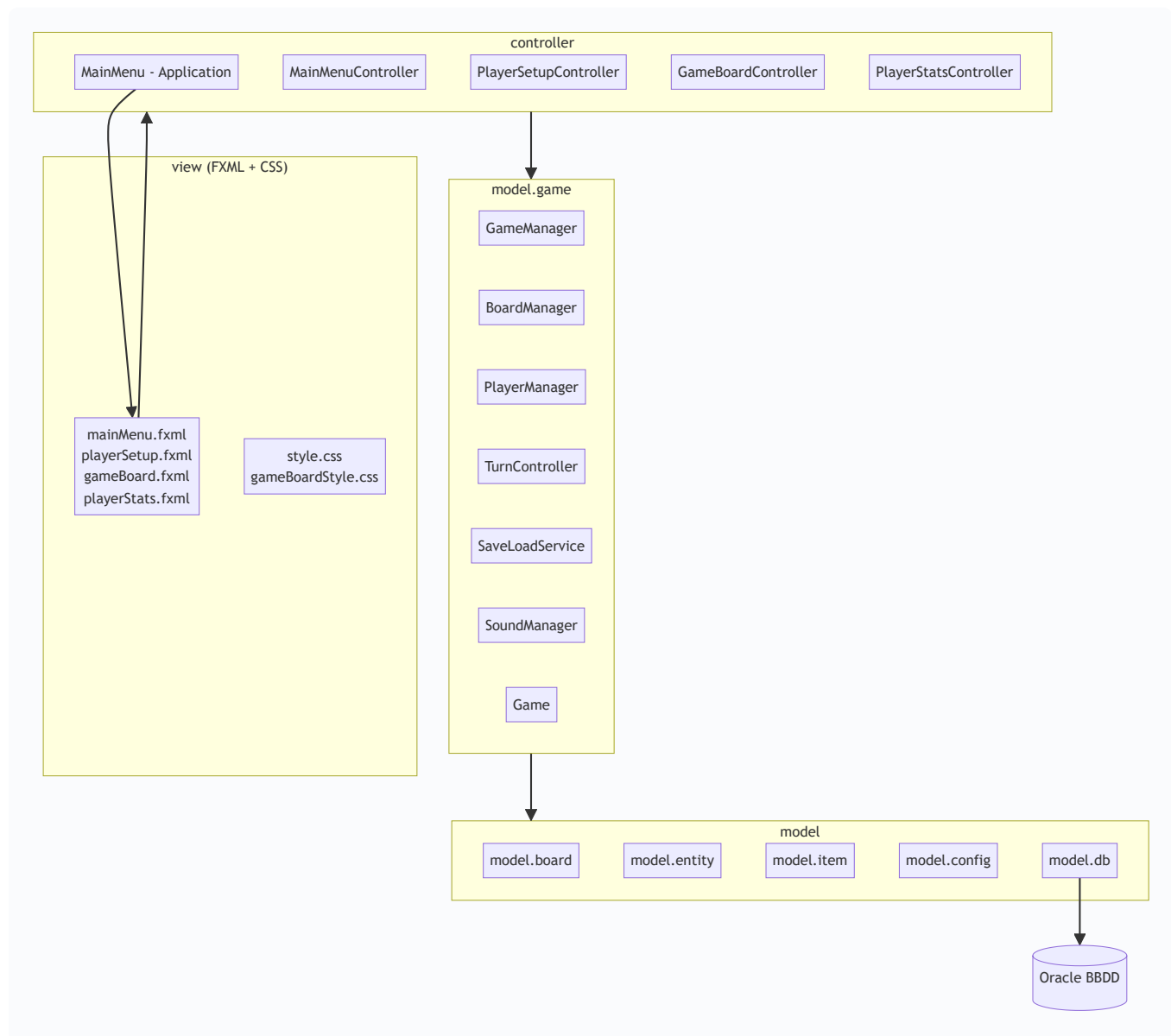
PART II - GUIA TÈCNICA

Arquitectura General

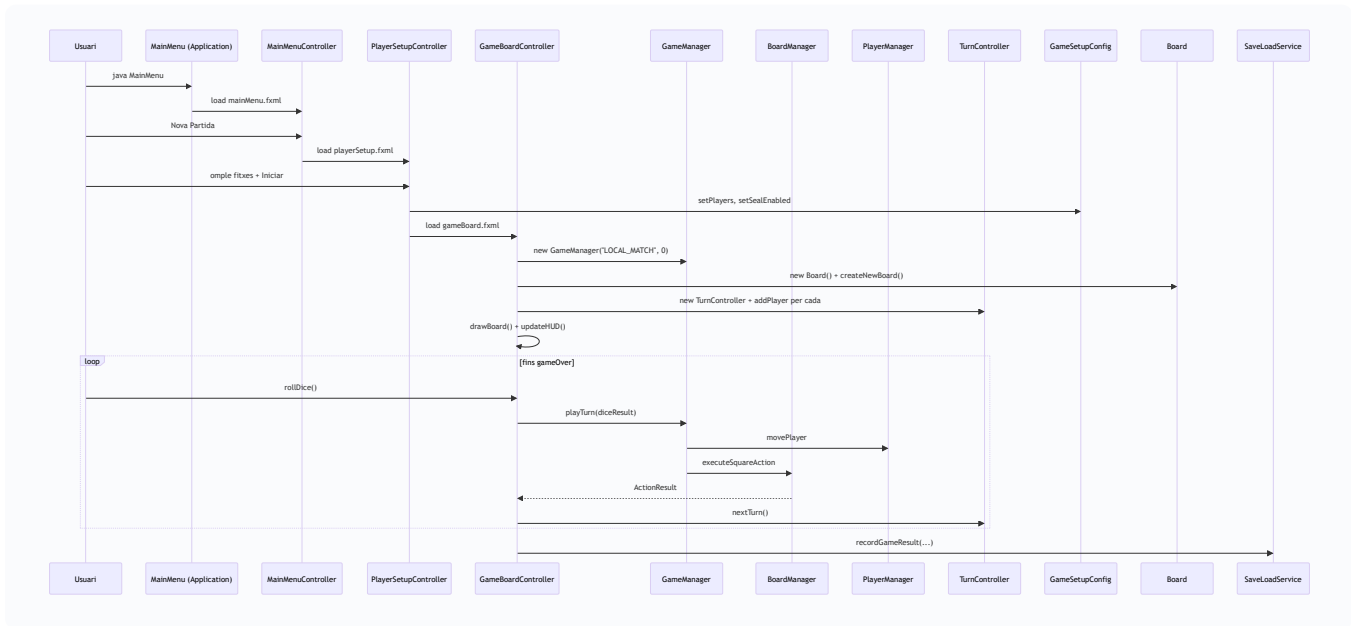
El projecte segueix una variant **MVC** clàssica adaptada a JavaFX amb FXML:

- **Model** — paquets `model.*` (lògica, dades, persistència).
- **View** — fitxers FXML + CSS sota `view.fxml/` i `assets/css/`.
- **Controller** — classes Java sota `controller.*` que connecten els FXML amb el model.

Vista de mòduls



Cicle de vida d'una partida



Capes i responsabilitats

Capa	Paquets	Responsabilitat
Vista	<code>view.fxml</code> , <code>assets/css</code>	Descripció declarativa de la UI
Controlador UI	Paquet <code>controller</code>	Bind FXML ↔ model, gestió d'events JavaFX
Coordinador	Paquet <code>model.game</code> (<code>GameManager</code> , <code>BoardManager</code> , <code>PlayerManager</code>)	Regles del joc
Estat del joc	Paquet <code>model.game</code> (<code>Game</code> , <code>TurnController</code>)	Estructures vives
Domini	Paquet <code>model.board</code> , Paquet <code>model.entity</code> , Paquet <code>model.item</code>	Tauler, jugadors, objectes
Configuració	Paquet <code>model.config</code>	Idiomes (<code>Lang</code> / <code>LangConfig</code>), criptografia (<code>CryptoUtil</code>), bag de setup (<code>GameSetupConfig</code>)
Persistència	Paquet <code>model.db</code> (BBDD) + <code>SaveLoadService</code>	Connexió Oracle + serialització YAML

Punts d'entrada

- `controller.main.MainMenu.main()`: bootstrap. Carrega l'idioma per defecte i invoca `Application.launch()`.
- `view.ui.BBDDPanel.main()`: utilitat secundària per provar les operacions CRUD contra Oracle (no és part del joc).

Patrons utilitzats

Patró	On
MVC	Global
Singleton	<code>LangConfig</code> , <code>SoundManager</code>
Template Method	<code>Square.action(player)</code> abstracte amb subclasses <code>S_*</code>
Strategy (via <i>switch</i>)	<code>BoardManager.executeSquareAction</code>
Observer / Listener	<code>LangConfig.addLanguageChangeListener</code>
Data Transfer Object	<code>ActionResult</code> (DTO entre lògica i UI)

Decisions de disseny destacades

PER QUÈ `GAMESETUPCONFIG` ÉS ESTÀTIC?

Els controllers JavaFX són instanciats pel `FXMLLoader`, així que passar estat directament entre `PlayerSetupController` i `GameBoardController` és incòmode. Una classe estàtica fa de "bossa" temporal.

UNA SOLA FINESTRA

Tot el joc viu en una **única Stage** que canvia d' `Scene` segons la pantalla. Això facilita la lògica de música (`SoundManager`) i el redimensionament.

CAP `*THREAD*` DE FONS

Tota la lògica corre al **JavaFX Application Thread**. Les animacions utilitzen `Timeline` / `PauseTransition`, no fils separats.

Enllaços relacionats

- Estructura de Paquets — mapa detallat de paquets
- Diagrama de Classes — UML general
- Flux de Joc — seqüències detallades

Estructura de Paquets

El projecte segueix una jerarquia clara. Cada paquet té una pàgina pròpia amb el detall.

```

src/
├─ module-info.java
├─ controller/
│   ├── main/MainMenu.java           ← entry point
│   └─ ui/                            ← controllers FXML
│       ├── MainMenuController.java
│       ├── PlayerSetupController.java
│       ├── GameBoardController.java  (≈1780 línies)
│       └─ PlayerStatsController.java
├─ model/
│   ├── board/                        ← tauler + caselles
│   │   ├── Board.java
│   │   ├── Square.java              (abstracte)
│   │   ├── SquareType.java          (enum)
│   │   ├── EventManager.java
│   │   └─ squares/
│   │       ├── S_Normal.java  S_Start.java  S_End.java
│   │       ├── S_Bear.java    S_IceHole.java
│   │       ├── S_Sled.java    S_BrokenFloor.java
│   │       └─ S_Event.java
│   ├── entity/                      ← pingüins + foca
│   │   ├── Entity.java              (abstracte)
│   │   ├── EntityType.java          (enum)
│   │   ├── Player.java
│   │   └─ Seal.java
│   ├── item/                        ← objectes / inventari
│   │   ├── GameObject.java          (abstracte)
│   │   ├── ObjectType.java          (enum)
│   │   ├── Inventory.java
│   │   └─ objects/
│   │       ├── Dice.java
│   │       ├── Fish.java
│   │       └─ SnowBall.java
│   ├── game/                        ← coordinadors / persistència
│   │   ├── Game.java
│   │   ├── GameManager.java
│   │   ├── BoardManager.java
│   │   ├── PlayerManager.java
│   │   ├── TurnController.java
│   │   ├── ActionResult.java
│   │   ├── SaveLoadService.java
│   │   └─ SoundManager.java
│   ├── config/                      ← localització + crypto + bag de setup
│   │   ├── Lang.java                (enum claus de traducció)
│   │   ├── LangConfig.java          (singleton de l'idioma)
│   │   ├── GameSetupConfig.java     (bag estàtic)
│   │   └─ CryptoUtil.java           (AES-128)
│   └─ db/
│       └─ BBDD.java                 (Oracle helper del curs)
├─ view/
│   ├── ui/BBDDPanel.java             (test manual de BBDD)
│   └─ fxml/
│       ├── mainMenu.fxml             ← MainMenuController
│       ├── playerSetup.fxml          ← PlayerSetupController
│       ├── gameBoard.fxml            ← GameBoardController
│       └─ playerStats.fxml           ← PlayerStatsController

```

```
└─ assets/
  │─ css/style.css · gameBoardStyle.css
  │─ lang/<codi>.yaml
  │─ sprites/...
  │─ sounds/main_screen_music.wav · bg_music.wav · dice.wav · ...
  └─ font/pixel-game.regular.otf · pixel-game.extrude.otf · pixel-unicode-regular.ttf
```

Mòdul Java (module-info.java)

```
module JocDelPingu {
    requires org.yaml.snakeyaml;
    requires java.sql;
    requires java.desktop;

    requires javafx.graphics; javafx.controls; javafx.fxml;
    requires javafx.swing;    javafx.base;    javafx.media;

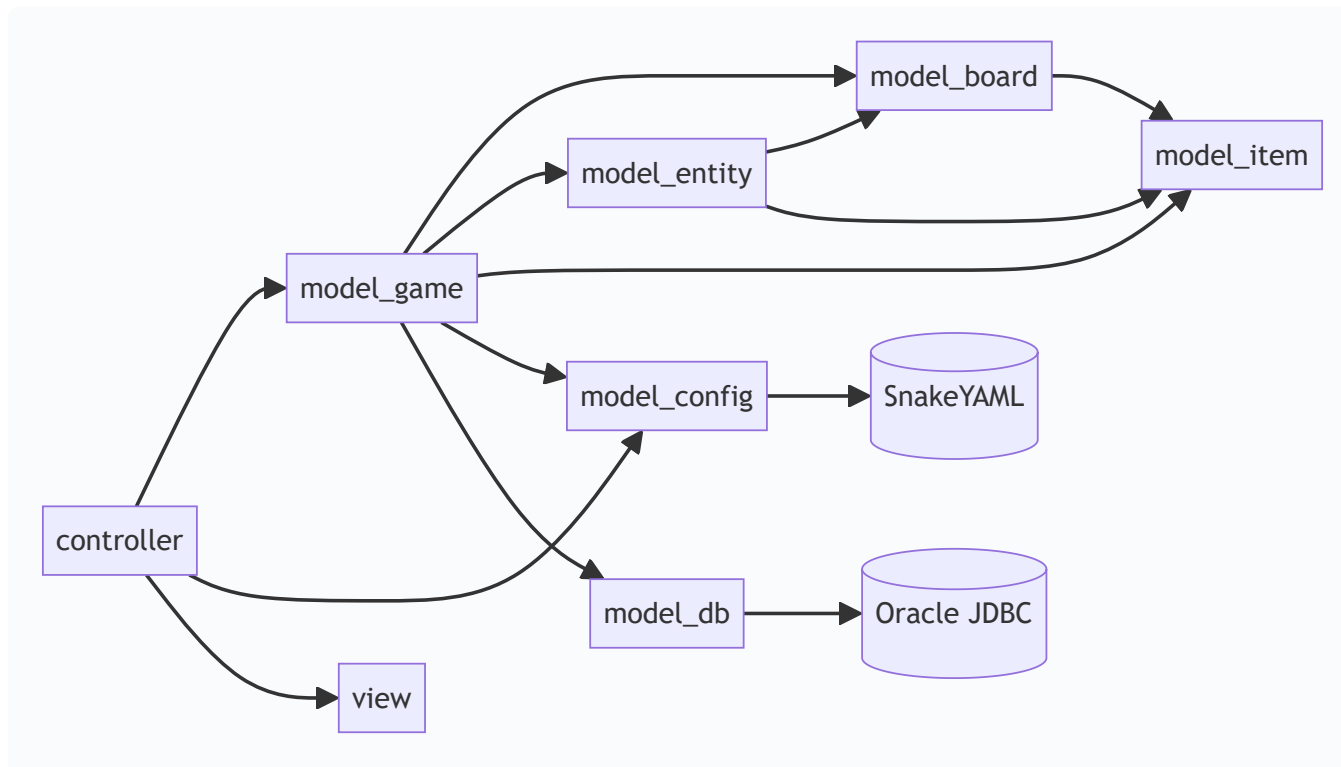
    opens controller.main to javafx.graphics, javafx.fxml;
    opens controller.ui   to javafx.fxml,    javafx.graphics;
    opens view.ui         to javafx.fxml,    javafx.graphics;
    opens view.fxml       to javafx.fxml,    javafx.graphics;
    opens assets.css      to javafx.graphics, javafx.fxml;

    exports controller.main; exports controller.ui; exports view.ui;
    exports model.board; exports model.board.squares;
    exports model.config; exports model.db;
    exports model.entity;
    exports model.game;
    exports model.item; exports model.item.objects;
}
```

‘OPENS’ VS ‘EXPORTS’

`opens` permet a JavaFX accedir per reflexió als controllers (necessari per a `@FXML`). `exports` permet l'accés normal des d'altres mòduls.

Relacions entre paquets



Pàgines detallades per paquet

Paquet	Pàgina
<code>model.board</code> (+ <code>.squares</code>)	Paquet model.board
<code>model.entity</code>	Paquet model.entity
<code>model.item</code> (+ <code>.objects</code>)	Paquet model.item
<code>model.game</code>	Paquet model.game
<code>model.config</code>	Paquet model.config
<code>model.db</code>	Paquet model.db
<code>controller.*</code>	Paquet controller
<code>view.*</code> + FXML + CSS	Paquet view

Enllaços relacionats

- [Arquitectura General](#) — visió MVC
- [Diagrama de Classes](#) — UML general

Paquet `model.board`

Conté la representació del tauler i totes les classes de casella. Subpaquet `.squares` per les implementacions concretes de cada tipus.

Classes principals



Syntax error in text
mermaid version 10.9.6

Board

Manté un array de 50 caselles i dues llistes auxiliars:

- `IceHole_Array` — índexs ordenats dels forats de gel. Es fa servir a `getDestination()` per buscar el forat anterior.
- `Sled_Array` — índexs ordenats dels trineus. Idèntic, però buscant el següent.

Mètodes destacats

Mètode	Què fa
<code>createNewBoard()</code>	Genera 50 caselles aleatòries (70% NORMAL, 30% especials). Posa <code>S_Start</code> a l'índex 0 i <code>S_End</code> a l'últim.
<code>loadBoard(types)</code>	Reconstrueix el tauler des d'una llista de noms d'enum (per a càrrega de partides desades).
<code>getDestination(idx)</code>	Per <code>ICE_HOLE</code> retorna el forat anterior; per <code>SLED</code> el següent trineu; sinó retorna el mateix idx.
<code>convertBrokenFloorToIceHole(idx)</code>	Quan algú cau a un <code>BROKEN_FLOOR</code> per excés d'objectes, la casella es transforma en un <code>ICE_HOLE</code> i s'insereix ordenat a <code>IceHole_Array</code> .
<code>dropEdgeIndices(list)</code>	Defensiu — elimina índexs 0/49 de la llista per evitar que START/END es resolguin com a hole/sled.

DISTRIBUCIÓ ALEATÒRIA

Per cada cel·la: si `random*100 + 1 <= 70` → NORMAL; sinó es tria 1-5 amb pesos iguals entre ICE_HOLE, SLED, BEAR, EVENT, BROKEN_FLOOR.

Square (abstracta)

Camps: `type: SquareType`, `SquareID: int`. Mètode abstracte:

```
public abstract String action(Player player);
```

Algunes subclasses retornen `null` per indicar "**resolució delegada al BoardManager**" (forat de gel, trineu, esdeveniment).

SquareType (enum)

Vegeu la taula completa a [Caselles Especials](#).

Subclasses `s_*`

Classe	Efecte de <code>action(player)</code>
<code>S_Normal</code>	Només produeix missatge ("X landed on a normal square")
<code>S_Start</code>	Missatge ("X is at the start")
<code>S_End</code>	Missatge de victòria amb 🎉
<code>S_Bear</code>	Si té peix: <code>useObject(FISH, 1)</code> i missatge "bribed". Sinó: <code>player.setSquare(0)</code> i missatge "attacked".
<code>S_IceHole</code>	Retorna <code>null</code> — gestionat per <code>BoardManager.handleIceHole</code>
<code>S_Sled</code>	Retorna <code>null</code> — gestionat per <code>BoardManager.handleSled</code>
<code>S_Event</code>	Retorna <code>null</code> — gestionat per <code>BoardManager.handleEvent</code>
<code>S_BrokenFloor</code>	Branca en 3 segons <code>getTotalItemCount()</code> : $>5 \rightarrow \text{start}$, $1-5 \rightarrow 50\% \text{ perdre item} / 50\% \text{ perdre torn}$, $0 \rightarrow \text{res}$

PER QUÈ `S\_EVENT.ACTION` ÉS BUIT?

Una versió antiga incloïa un randomitzador 1-of-6 que duplicava la lògica de l' `EventManager` (amb probabilitats incorrectes). Ara la resolució viu **només** a `EventManager.triggerEvent()` per garantir les probabilitats correctes (30/20/15/12/10/8/5).

EventManager

Subsistema d'esdeveniments aleatoris per caselles `EVENT`. Veure [Caselles Especials#EVENT](#) per a probabilitats.

Enum intern `EventType`

Constant	Missatge anglès
<code>GET_FISH</code>	"🐟 You found a fish!"
<code>GET_SNOWBALLS</code>	"🧊 You found snowballs!"
<code>GET_FAST_DICE</code>	"🎲🚀 You found a FAST die!"
<code>GET_SLOW_DICE</code>	"🎲🐢 You found a slow die"
<code>LOSE_TURN</code>	"❄️ You slipped on ice!"
<code>LOSE_ITEM</code>	"💨 A gust of wind blew away one of your items!"
<code>SNOWMOBILE</code>	"🚗❄️ You found a snowmobile!"

Classe interna `EventResult`

DTO amb camps: `type`, `message`, `detail`, `newPosition` (-1 si la posició no canvia).

```
public static EventResult triggerEvent(Player player, Board board) {
    int roll = random.nextInt(100);
    if (roll < 30)      return handleGetSlowDice(player);
    else if (roll < 50) return handleGetSnowballs(player);
    // ...
}
```

Enllaços relacionats

- [Caselles Especials](#) — vista d'usuari
- [Paquet model.game](#) — `BoardManager` orquestra els handlers
- [Paquet model.entity](#) — `Player` rep els efectes
- [Diagrama de Classes](#) — UML global

Conté les entitats que es mouen pel tauler: el **Player** (humà) i la **Sea1** (CPU opcional), amb una superclasse comuna **Entity**.

Classes



Syntax error in text
mermaid version 10.9.6

Entity (abstracta)

Comporta tot l'estat genèric:

Camp	Tipus	Significat
entityId	int	ID aleatori (no UUID)
numSquare	int	Índex actual al tauler (0..49)
type	EntityType	PLAYER o SEAL
board	Board	Referència al tauler (transient)
skipNextTurn	boolean	Marca si ha de perdre el següent torn
facingRight	boolean	Estat visual transient
damaged	boolean	Mostra sprite de dany (flash)
frozen	boolean	Mostra sprite congelat
lastEvent	EventResult	Últim event que ha tocat (per UI)

Mètodes importants

- advance(steps)**: avança N caselles. Retorna **true** si ha arribat al final. Actualitza **facingRight** segons direcció.
- setSquare(newPosition)**: clampa a **[0, MAX_SQUARES-1]** i actualitza **facingRight**.
- setNumSquare(int)**: igual però sense clampar manualment.

Player

Subclasse principal. Constructors:

```
public Player(String name, String colour);  
public Player(String name, String colour, String password);
```

Camps específics:

- `name`, `colour` (hex de 6 dígits), `password` (sense encriptar en memòria).
- `avatarPath`: URI a una imatge personalitzada (opcional).
- `inventory`: instància d'`Inventory`.
- `eventHistory`: `List<String>` capada a `MAX_EVENT_HISTORY = 50` entrades.

Mètodes específics

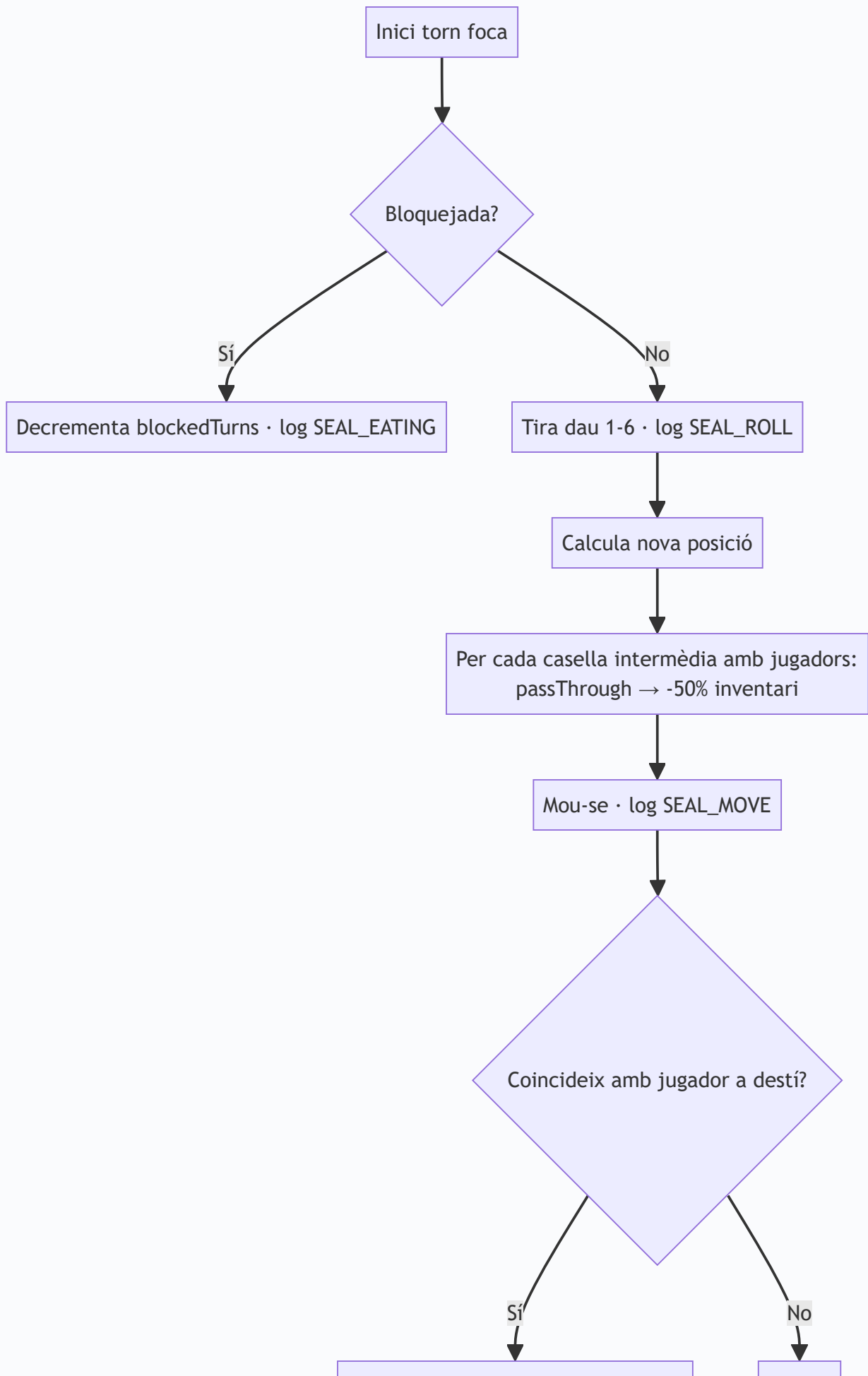
- `recordEvent(msg)`: afegeix al log personal i descarta les més antigues si supera 50.
- `loseHalfInventory()`: delega a `inventory.removeHalf()`. Es crida quan la foca passa per sobre del jugador.

Seal (CPU)

Una sola foca per partida. Estat:

- `blockedTurns` — quants torns està menjant un peix.
- `random` — generador d'aleatoris per la seva tirada.

Comportement durant `playTurn(allPlayers)`



hitPlayer → enviar al forat anterior

F1 torn

Mètodes destacats

Mètode	Què fa
<code>interact(player)</code>	Quan un jugador cau a la casella de la foca. Si bloquejada → safe. Si té peix → <code>bribeSeal</code> . Sinó → <code>hitPlayer</code> .
<code>bribeSeal(player)</code>	Gasta 1 peix → bloqueja 2 torns la foca
<code>hitPlayer(player)</code>	Envia el jugador al forat de gel anterior (o casella 0)
<code>passThrough(player)</code>	Crida <code>player.loseHalfInventory()</code>
<code>updateSealTurns()</code>	Decrementa <code>blockedTurns</code> ; al arribar a 0 emet <code>SEAL_ACTIVE</code>
<code>playTurn(allPlayers)</code>	Bucle complet: tira, mou, processa col·lisions
<code>findPreviousIceHole(pos)</code>	Recorre <code>board.getIceHole_Array()</code> per trobar el forat anterior

VICTÒRIA DE LA FOCA

Si la foca arriba a `Board.MAX_SQUARES - 1` (índex 49), TOTS els jugadors humans perden. Es marca a `GameBoardController.playSealTurnAnimated()` amb `gameOver = true` i `winner = "Seal"`.

EntityType (enum)

```
public enum EntityType { PLAYER, SEAL }
```

Es fa servir per diferenciar entitats sense fer `instanceof` repetidament al codi de joc.

Enllaços relacionats

- Paquet `model.board` — taula on es mouen
- Paquet `model.item` — inventari del jugador
- Paquet `model.game` — `ActionResult` reporta efectes de la foca
- Sistema de Sprites — com es dibuixen `damaged` / `frozen` / `facingRight`

Modela els objectes de l'inventari (peixos, boles, daus) i la classe `Inventory` que els emmagatzema. Subpaquet `.objects` per a les implementacions concretes.

Diagrama de classes



Syntax error in text
mermaid version 10.9.6

`GameObject` (abstracta)

Plantilla mínima per tot objecte. Camps:

- `objectId`: ID aleatori (només per logs).
- `name`: etiqueta humana.
- `type`: `ObjectType`.
- `quantity`: poc usat — la majoria de comptadors viuen a `Inventory`.

Constructor: assigna un id aleatori amb `Math.random()*100000`.

`ObjectType` (enum)

Constant	Significat
<code>SNOWBALL</code>	Bola de neu (ofensiva)
<code>FISH</code>	Peix (subornar ós o foca)
<code>DICE</code>	Marker genèric / comptador total de daus
<code>FASTDICE</code>	Dau de 5-10
<code>SLOWDICE</code>	Dau d'1-3

``DICE`` ÉS UN SUMATORI

Es manté com a sub-tipus per comoditat (`diceQuantity = fastdiceQuantity + slowdiceQuantity`), però mai s'instancia un objecte `Dice` amb tipus `DICE` a la partida — només `FASTDICE` i `SLOWDICE` acaben a l'inventari.

Dice

```
public Dice(ObjectType diceType) {
    super(diceType);
    switch (diceType) {
        case FASTDICE:
            this.setName("Fast Dice");
            this.minValue = 5; this.maxValue = 10;
            break;
        case SLOWDICE:
            this.setName("Slow Dice");
            this.minValue = 1; this.maxValue = 3;
            break;
        default:
            this.setName("Dice");
            this.minValue = 1; this.maxValue = 6;
    }
}

public int roll() {
    return (int) ((Math.random() * (maxValue - minValue + 1)) + minValue);
}
```

GameBoardController crea **tres instàncies** de `Dice` (default, fast, slow) i les reutilitza a totes les tirades.

Fish i SnowBall

Subclasses trivials que només estableixen el nom i el tipus al constructor:

```
public Fish() { super(ObjectType.FISH); setName("Fish"); }
public SnowBall(){ super(ObjectType.SNOWBALL); setName("Snowball"); }
```

NO S'INSTANCIEN GAIREBÉ MAI

Tot i existir, aquestes classes pràcticament no es creen com a objectes durant el joc — `Inventory` només manté **comptadors** entera. Existeixen per coherència amb el patró `GameObject` (per si calgués estendre el sistema d'objectes).

Inventory

És el cor del sistema d'objectes. Estat:

Camp	Màxim	Comentari
snowballQuantity	6 (MAX_SNOWBALLS)	
fishQuantity	2 (MAX_FISH)	
diceQuantity	3 (MAX_DICE)	Suma de fastdice + slowdice
fastdiceQuantity	—	Part de diceQuantity
slowdiceQuantity	—	Part de diceQuantity

API resumida

```

int  addSnowballs(int count);           // afegeix fins a cap
boolean addFish();                     // true si afegit
boolean addDice(ObjectType diceType);  // FASTDICE o SLOWDICE
int  getObjectQuantity(ObjectType o);
boolean useObject(ObjectType o, int n); // false si no hi ha prou
ObjectType removeRandomItem();         // null si buit
void removeHalf();                     // foca passant per sobre
int  getTotalItemCount();              // snowball + fish + fast + slow

```

REMOVEHALF()

Divideix per 2 (enter) cada comptador. diceQuantity es recalcula com fastdiceQuantity + slowdiceQuantity per evitar inconsistències.

Casos d'ús al joc

- **Casella BEAR:** `inv.useObject(FISH, 1)` si en té; sinó `setSquare(0)`.
- **Casella EVENT GET_X:** `inv.addSnowballs(n)`, `inv.addFish()`, `inv.addDice(type)`.
- **Casella EVENT LOSE_ITEM:** `inv.removeRandomItem()`.
- **Casella BROKEN_FLOOR > 5 items:** `setSquare(0)` (sense tocar inventari).
- **Foca passant:** `inv.removeHalf()`.
- **Snowball war:** `useObject(SNOWBALL, balls)` per a tots dos.

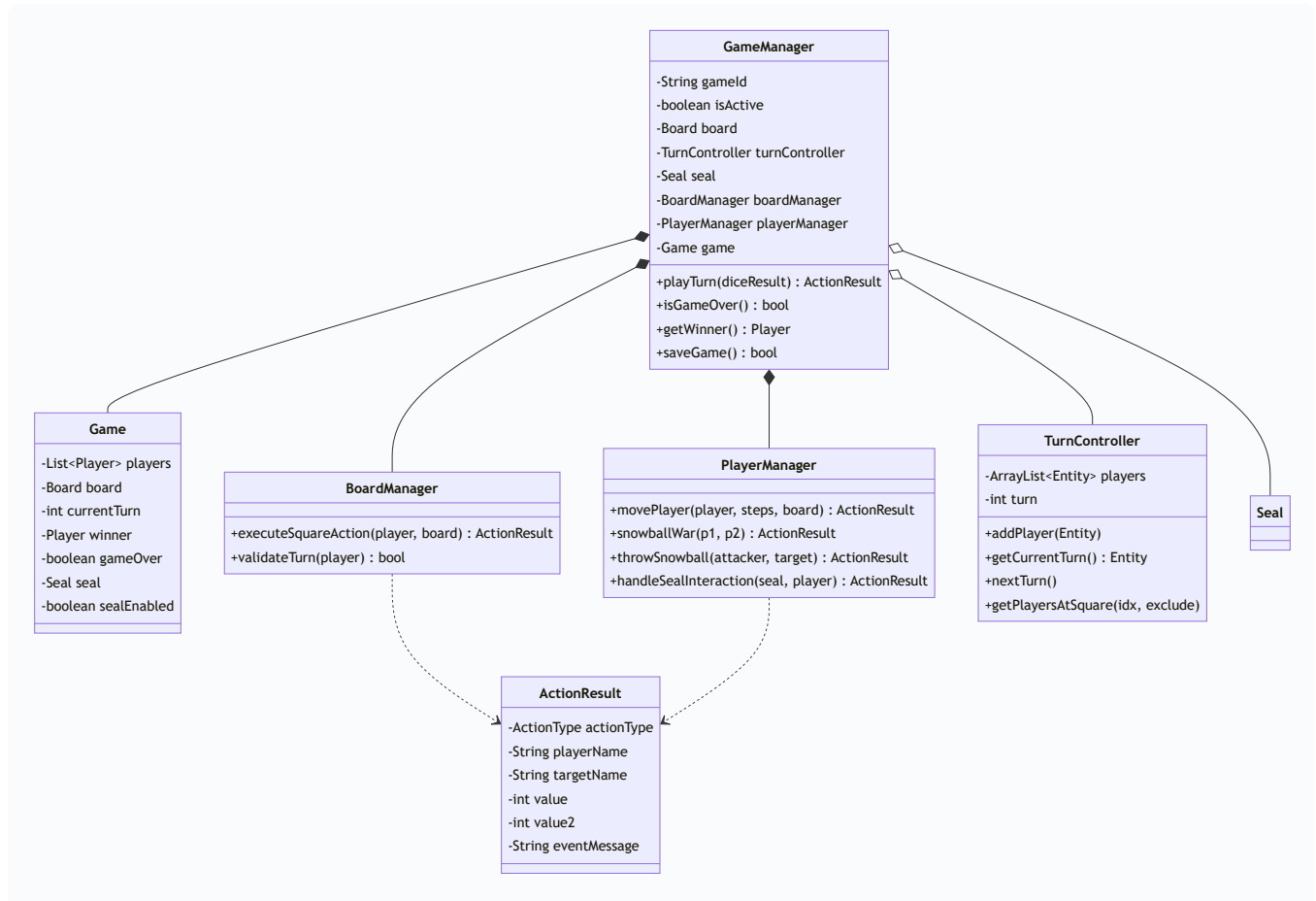
Enllaços relacionats

- [Inventari i Objectes](#) — vista d'usuari
- [Paquet model.entity](#) — `Player.inventory`
- [Caselles Especials](#) — quan es modifica

Paquet `model.game`

Aquest paquet conté el **motor del joc** (coordinadors + estat agregat) i el servei de persistència. És el "cervell" que orquestra el moviment de jugadors i la resolució de caselles.

Visió general



Game

POJO d'estat agregat (DTO mutable). Té setters/getters per a tot:

Camp	Tipus
<code>players</code>	<code>List<Player></code>
<code>board</code>	<code>Board</code>
<code>currentTurn</code>	<code>int</code>
<code>winner</code>	<code>Player</code> (null mentre la partida està en curs)
<code>gameOver</code>	<code>boolean</code>
<code>seal</code>	<code>Seal</code> (opcional)
<code>sealEnabled</code>	<code>boolean</code>

PER QUÈ POJO SENSE LòGICA?

SnakeYAML pot serialitzar directament aquesta classe, i els coordinadors la modifiquen com un *bag*. Tota la lògica viu a les classes **Manager*.

GameManager

Punt central. Mètode clau:

```
public ActionResult playTurn(int diceResult) {
    Player current = getCurrentPlayer();
    ActionResult moveMsg = playerManager.movePlayer(current, diceResult, board);
    if (moveMsg != null) { // jugador ha arribat al final
        game.setGameOver(true);
        game.setWinner(current);
        return moveMsg;
    }
    return boardManager.executeSquareAction(current, board);
}
```

CONSTRUCCIÓ DISPERSA

Al constructor només es crea `boardManager`, `playerManager` i `game`. El `Board`, `TurnController` i `Seal` s'**injecten** via setters des de `GameBoardController.initialize()`.

BoardManager

Resol l'efecte d'aterrar a una casella concreta. El mètode `executeSquareAction(player, board)` fa un *switch* sobre `SquareType`:

Tipus	Mètode privat	Resultat
ICE_HOLE	<code>handleIceHole</code>	mou al forat anterior
SLED	<code>handleSled</code>	mou al següent trineu
BEAR	<code>handleBear</code>	gasta peix o torna a 0
EVENT	<code>handleEvent</code>	crida <code>EventManager.triggerEvent</code>
BROKEN_FLOOR	<code>handleBrokenFloor</code>	3 branques segons items totals
Altres	<code>inline</code>	retorna <code>ActionResult</code> simple

També té `validateTurn(player)` que consumeix el flag `skipNextTurn`.

PlayerManager

Maneja moviment i interaccions entre jugadors.

Mètode	Què fa
<code>movePlayer(p, steps, board)</code>	Avança steps caselles. Retorna <code>ActionResult WIN</code> si arriba al final, sinó <code>null</code> .
<code>snowballWar(p1, p2)</code>	Tots dos gasten totes les boles → qui té més guanya → l'altre retrocedeix per la diferència. Empat: ningú es mou.
<code>throwSnowball(attacker, target)</code>	Atacant gasta 1 bola → objectiu retrocedeix 1-3 caselles aleatori.
<code>handleSealInteraction(seal, p)</code>	Delega a <code>seal.interact(p)</code> .

TurnController

Round-robin amb `ArrayList<Entity>`. Mètodes:

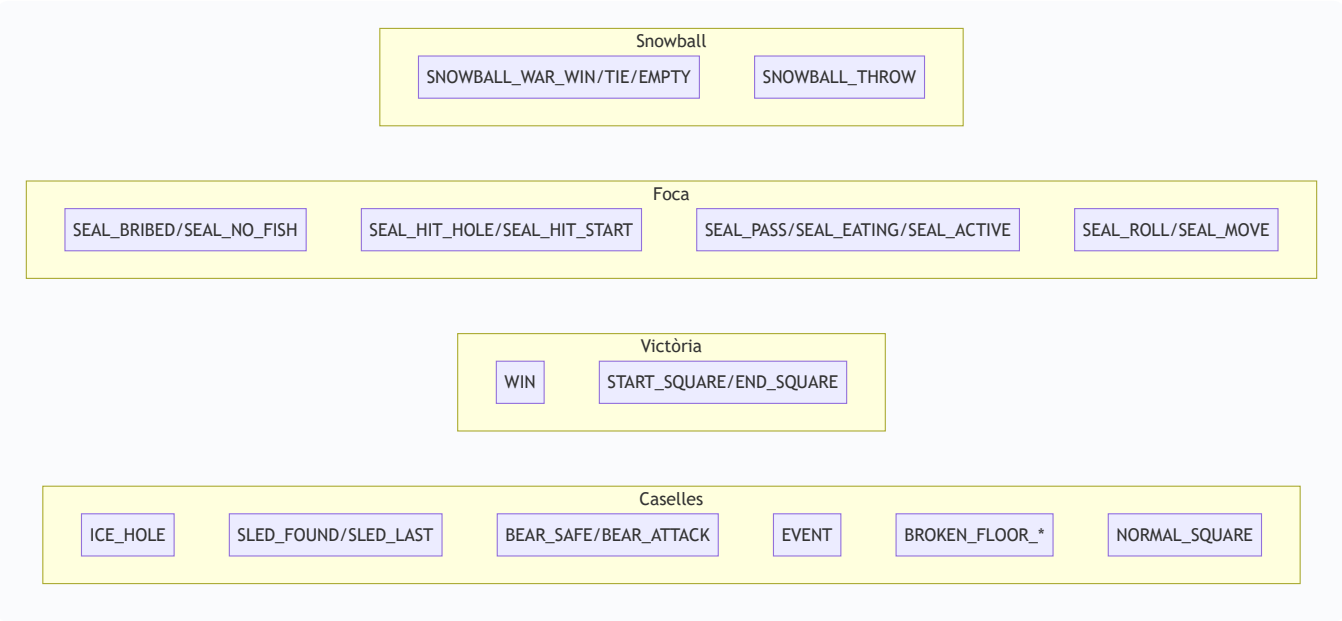
Mètode	Comentari
<code>addPlayer(Entity)</code>	Afegeix a la cua (humà o foca)
<code>getCurrentTurn()</code>	<code>players.get(turn)</code>
<code>nextTurn()</code>	<code>turn = (turn+1) % size</code> . Si el següent té <code>shouldSkipNextTurn()==true</code> , consumeix el flag i salta un altre
<code>getHumanPlayers()</code>	Filtra només els <code>Player</code> (sense la foca)
<code>getPlayersAtSquare(idx, excl)</code>	Per detectar col·lisions snowball war

ActionResult (DTO)

DTO que viatja del motor a la UI. Té un `ActionType` enum amb 25+ variants i camps genèrics:

- `playerName` / `targetName`
- `value` / `value2` (per dades numèriques: índex destinació, dau, comptadors)
- `eventMessage` (per text custom d'events)

ActionType **agrupat**



GameBoardController.formatActionMessage tradueix ActionType a una cadena humana per al log.

SaveLoadService

Servei estàtic de persistència contra Oracle.

Mètodes públics

Mètode	Funció
saveGame(name, board, tc, seal, winner)	Serialitza a YAML, encripta amb CryptoUtil, INSERT a SAVED_GAMES (CLOB)
loadGame(gameId)	SELECT, desencripta, parseja YAML, omple GameSetupConfig
getAllSavedGameIds()	Per al ChoiceDialog de carregar
recordGameResult(allPlayers, winnerName)	INSERT a GAME + UPDATE counters a ENTITY
registerPlayer(name, pwd, color)	MERGE a ENTITY (insert/update) amb contrasenya encriptada
verifyPassword(name, input)	Compara contrasenya escrita contra el valor desencriptat de BBDD
getPlayerStats()	SELECT per a la pantalla d'estadístiques
getRegisteredPlayers()	SELECT per al picker "seleccionar jugador existent"

Detalls a Sistema de Persistència.

SoundManager **(singleton)**

Gestiona música i SFX. Veure Sistema de So per a detalls.

Enllaços relacionats

- [Arquitectura General](#) — visió MVC
- [Paquet model.board](#) · [Paquet model.entity](#) · [Paquet model.item](#)
- [Paquet controller](#) — qui crida tot això
- [Sistema de Persistència](#) · [Sistema de So](#)
- [Flux de Joc](#) — diagrames de seqüència

Conté 4 classes amb responsabilitats transversals: localització (`Lang` / `LangConfig`), bag de setup (`GameSetupConfig`) i criptografia (`CryptoUtil`).

Classes



Syntax error in text

mermaid version 10.9.6

Lang (enum)

Mapeja constants Java a claus YAML. Agrupat per àrea:

```
ENTITY_SEAL("entity.seal"),
OBJECT_SNOWBALL("object.snowball"),
MENU_BUTTON_NEWGAME("menu.button.newgame"),
ALERT_WRONGPASSWORD_TITLE("alert.wrongpassword.title"),
DIALOG_SELECTPLAYER_HEADER("dialog.selectplayer.header");
```

Avantatge: typos al codi es detecten en compilació en lloc de retornar `null` en temps d'execució.

LangConfig (singleton)

Carrega un YAML i exposa una API per consultar traduccions.

Estructura interna

Camp	Tipus	Significat
<code>LANG_DIR</code>	<code>String</code>	<code>/assets/lang/</code>
<code>DEFAULT_LANG</code>	<code>String</code>	<code>"en"</code>
<code>AVAILABLE_LANGUAGES</code>	<code>String[]</code>	<code>{en, es, ca, fr, pt, ro, ar, ru, uk, ff, jp}</code>
<code>LANGUAGE_DISPLAY_NAMES</code>	<code>LinkedHashMap</code>	<code>"🌐 English", "🌐 Català", ...</code>
<code>data</code>	<code>Map<String,String></code>	Diccionari clau→traducció (carregat de YAML)
<code>listeners</code>	<code>List<Runnable></code>	Callbacks dels controllers per refrescar UI

API principal

```
LangConfig.loadLang();           // carrega "en.yml"
LangConfig.loadLang("ca");       // canvi d'idioma
String t = LangConfig.getLang(Lang.MENU_BUTTON_NEWGAME);
LangConfig.addLanguageChangeListener(this::refreshTexts);
String next = LangConfig.cycleLanguage(); // rotació al següent codi
```

Veure [Sistema d'Idiomes](#) per a la implementació detallada del patró listener.

Fallback

Si el fitxer demanat no existeix, intenta carregar `en.yml`. Si tampoc existeix, deixa la UI sense traduir però **no llança excepció** — el `getLang()` retorna la pròpia clau (e.g. `"menu.button.newgame"`) per fer els errors visibles.

GameSetupConfig (bag estàtic)

Hand-off entre controllers. **Tots els camps són static**.

Camps

Camp	Quan s'escriu	Qui ho llegeix
<code>players</code>	<code>PlayerSetupController.handleStartGame()</code> o <code>SaveLoadService.loadGame()</code>	<code>GameBoardController.initializePlayers()</code>
<code>sealEnabled</code>	<code>PlayerSetupController</code>	<code>GameBoardController</code>
<code>isLoadingGame</code>	<code>MainMenuController</code> / setup controllers	<code>GameBoardController.initialize()</code>
<code>loadedBoardState</code>	<code>SaveLoadService.loadGame()</code>	<code>GameBoardController</code> (rehidrata Board)
<code>loadedTurnIndex</code>	<code>SaveLoadService.loadGame()</code>	<code>GameBoardController</code> (rehidrata TurnController)
<code>loadedSealState</code>	<code>SaveLoadService.loadGame()</code>	<code>GameBoardController</code> (rehidrata Seal)

ESTAT GLOBAL MUTABLE

És un anti-pattern en general (testejabilitat), però aquí funciona perquè els JavaFX controllers no es poden construir manualment. Per a una refactor més neta caldria usar `FXMLLoader.setControllerFactory()`.

CryptoUtil

Wrapper sobre **AES-128 amb una clau hardcoded de 16 bytes**.

```
private static final String ALGORITHM = "AES";
private static final byte[] KEY = "PinguGameKey1234".getBytes(); // 16 bytes
```


`encrypt(value)`

1. Crea `SecretKeySpec` amb la clau.
2. `Cipher.getInstance("AES")` → mode/padding per defecte de JCE (típicament `AES/ECB/PKCS5Padding`).
3. `cipher.doFinal(value.getBytes("UTF-8"))` → bytes encriptats.
4. Retorna `Base64.encodeToString(...)`.

`decrypt(encryptedValue)`

Inverteix: Base64 decode → AES decrypt → UTF-8 string.

LIMITACIONS DE SEGURETAT

- **ECB sense IV:** dos textos iguals produeixen el mateix ciphertext (vulnerable a anàlisi de patrons).
- **Clau hardcoded:** qualsevol decompilador la pot extreure.
- **No autènticat:** no protegeix contra manipulació.

Suficient per ofuscar contrasenyes en un projecte didàctic local. **No** apte per a producció.

Usos al codi

- Encriptar contrasenyes a `SaveLoadService.registerPlayer()` abans d'inserir-les a `ENTITY.PLAYERPASSWORD`.
- Verificar contrasenyes a `SaveLoadService.verifyPassword()`.
- Encriptar el blob YAML complet a `SaveLoadService.saveGame()` (camp `SAVED_GAMES.GAME_DATA`).

Enllaços relacionats

- Sistema d'Idiomes · Sistema de Persistència
- Idiomes i Localització — vista d'usuari
- Paquet `model.game` — `SaveLoadService` usa `CryptoUtil`
- Paquet `controller` — controllers que registren listeners

Conté `BBDD.java`, la classe d'utilitat per connectar amb la base de dades Oracle del curs. És una plantilla proporcionada pels professors d'Illerna (els comentaris originals són en castellà).

CODI DEL CURS

Bona part del codi conserva l'estil i els comentaris originals del professor — no s'ha refactoritzat per mantenir la coherència amb la resta de pràctiques de l'aula.

Connexió

```
public static Connection conectarBaseDatos(Scanner scan) {
    String entorno = "fuera"; // o "centro"
    String url = entorno.equals("centro")
        ? "jdbc:oracle:thin:@//192.168.3.26:1521/XEPDB2"
        : "jdbc:oracle:thin:@//oracle.illerna.com:1521/XEPDB2";

    String user = "DW2526_GR06_PINGU";
    String pwd = "ABDJFMV";

    Class.forName("oracle.jdbc.driver.OracleDriver");
    Connection con = DriverManager.getConnection(url, user, pwd);
    return con;
}
```

CREDENCIALS HARDCODED

L'usuari i la contrasenya estan al codi font. Per a un projecte didàctic és acceptable; **no** ho seria a producció.

Mètodes oferts

Mètode	Tipus	Què fa
<code>conectarBaseDatos(scan)</code>	static	Estableix la connexió i la retorna
<code>cerrar(con)</code>	static	Tanca la connexió ignorant excepcions
<code>insert(con, sql)</code>	static	INSERT (retorna files afectades)
<code>update(con, sql)</code>	static	UPDATE
<code>delete(con, sql)</code>	static	DELETE
<code>select(con, sql)</code>	static	SELECT → <code>ArrayList<LinkedHashMap<String,String>></code> (columnes → valors)
<code>print(con, sql, cols)</code>	static	SELECT debug (imprimeix per consola)
<code>executeInsUpDel(con, sql, label)</code>	static	Wrapper comú per a INSERT/UPDATE/DELETE

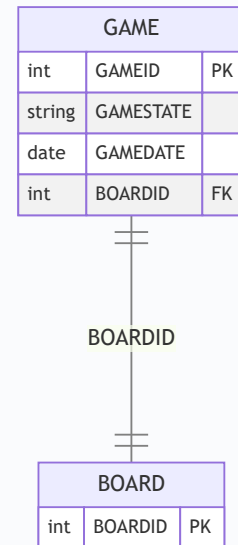
A més, dos mètodes d'instància/static legacy:

- `registrarJugadorEnBD(...)` — inserta a `ENTITY` directament (no encripta). Substituit per `SaveLoadService.registerPlayer`.
- `guardarPartida(listaJugadores)` — versió antiga del save (Base64, no AES). Substituïda per `SaveLoadService.saveGame`.

Esquema esperat a Oracle

ENTITY		
int	ENTITYID	PK
string	ENTITYTYPE	
string	PLAYERNAME	
string	PLAYERPASSWORD	
string	COLOUR	
int	GAMES_PLAYED	
int	GAMES_WON	

SAVED_GAMES		
string	GAME_ID	PK
CLOB	GAME_DATA	



ESQUEMA GESTIONAT PEL CURS

El DDL no està al projecte (es proporciona a l'aula). Aquesta documentació l'**infereix** dels usos a `SaveLoadService`.

Operacions típiques



Syntax error in text
mermaid version 10.9.6

BBDDPanel (test manual)

A `view.ui` hi ha `BBDDPanel.java` amb un `main()` que prova INSERT/UPDATE/DELETE manual sobre la taula `ENTITY`. No forma part del flux del joc, és només un test d'integració utilitzable des d'Eclipse.

Consideracions

Problema	Per què ho fa	Què caldria a producció
Concatenació SQL	Codi de curs senzill	PreparedStatement per a tot
<code>.replace("'", "'')</code>	Mitigació mínima d'injection	Bind parameters
Credencials hardcoded	Pràctica didàctica	Variables d'entorn o vault
Sense pooling	No cal a un joc local	HikariCP / connection pool

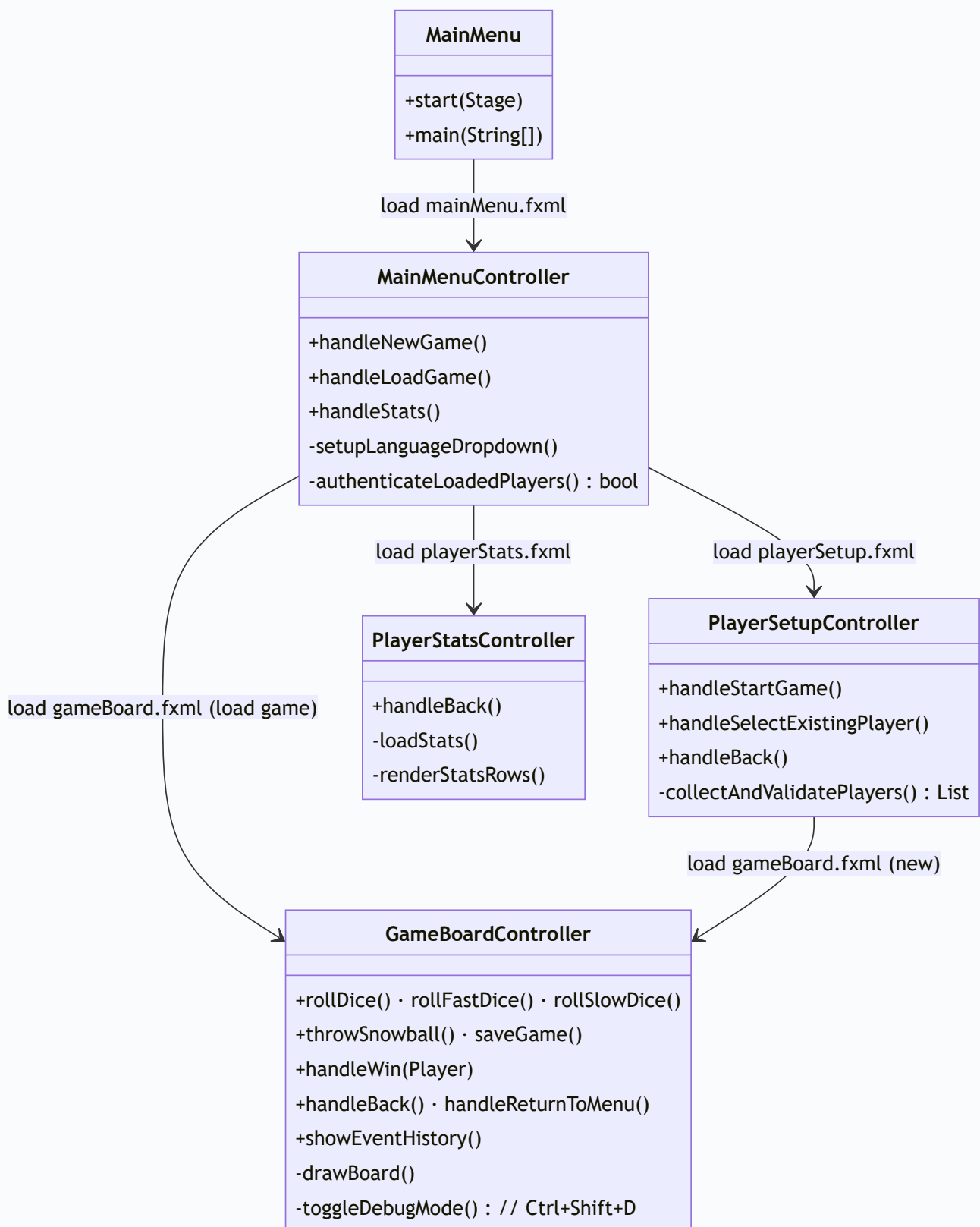
Enllaços relacionats

- Sistema de Persistència — fluxos complets
- Paquet model.game — SaveLoadService és qui crida BBDD.*
- Guardar i Carregar — vista d'usuari

Paquet `controller`

Conté els *controllers* JavaFX (subpaquet `controller.ui`) i el punt d'entrada de l'aplicació (`controller.main.MainMenu`).

Classes



MainMenu (Application)

És la classe `extends Application` que aixeca la finestra:

```
public static void main(String[] args) {
    LangConfig.loadLang();    // carrega l'idioma per defecte
    launch(args);
}

@Override
public void start(Stage primaryStage) throws Exception {
    // Carrega 3 fonts pixel-art
    Font.loadFont(... "/assets/font/pixel-game.regular.otf", 10);
    Font.loadFont(... "/assets/font/pixel-game.extrude.otf", 10);
    Font.loadFont(... "/assets/font/pixel-unicode-regular.ttf", 10);

    FXMLLoader loader = new FXMLLoader(getClass().getResource("/view/fxml/mainMenu.fxml"));
    Parent root = loader.load();
    Scene scene = new Scene(root);
    scene.getStylesheets().add(getClass().getResource("/assets/css/style.css").toExternalForm());

    primaryStage.setScene(scene);
    primaryStage.setWidth(900); primaryStage.setHeight(720);
    primaryStage.setResizable(true);
    primaryStage.setTitle(LangConfig.getLang(Lang.TEXT_GAME_TITLE));
    primaryStage.show();
}
```

Gestiona també el **fullscreen** preservant les dimensions originals.

MainMenuController

Lligat a `mainMenu.fxml` (StackPane amb VBox de botons + ComboBox d'idioma).

Camps @FXML

```
@FXML private StackPane rootPane;
@FXML private Button newGame_button, loadGame_button, stats_button;
@FXML private ComboBox<String> language_combobox;
@FXML private Text titleText, titleShadowText;
```

Mètodes destacats

- **initialize()**: posa imatge de fons, registra listener d'idioma, configura el combobox d'idiomes, inicia `SoundManager.playTitleMusic()`.
- **handleNewGame()**: carrega `playerSetup.fxml`.
- **handleLoadGame()**: `SaveLoadService.getAllSavedGameIds()` → `ChoiceDialog` → `loadGame()` → autenticació.
- **authenticateLoadedPlayers()**: bucle de `Dialog<String>` amb `PasswordField` per a cada jugador. Verifica amb `verifyPassword`. Si una falla → torna al menú i `setLoadedGame(false)`.
- **handleStats()**: carrega `playerStats.fxml`.

- `refreshTexts()`: callback registrat a `LangConfig` — actualitza tots els labels quan canvia l'idioma.

PlayerSetupController

Lligat a `playerSetup.fxml`. Gestió de la pantalla de configuració (1-4 jugadors + foca).

Camps @FXML rellevants

```
@FXML private ComboBox<Integer> numPlayersCombo;  
@FXML private CheckBox sealCheckBox;  
@FXML private VBox playersContainer;
```

I un `List<PlayerInput>` intern, on `PlayerInput` és una classe estàtica interna amb `nameField`, `passwordField`, `colorPicker`, `avatarPath`, `avatarBtn` i el seu `VBox` contenidor.

Mètodes destacats

- `updatePlayerFields(n)`: regenera les fitxes quan canvia el nombre.
- `createPlayerInput(num)`: construeix una fitxa amb `TextField`, `PasswordField`, `ColorPicker`, botó d'avatar (que obre `FileChooser`).
- `handleStartGame()`: valida contrasenyes amb `SaveLoadService.verifyPassword`, omple `GameSetupConfig` i carrega `gameBoard.fxml`.
- `handleSelectExistingPlayer()`: `ChoiceDialog` amb els jugadors ja existents a `ENTITY` → omple la primera fitxa buida.

PlayerStatsController

Lligat a `playerStats.fxml`. Mostra la classificació amb medalles (🥇 🥈 🥉) per als 3 primers:

```
private void loadStats() {  
    ArrayList<LinkedHashMap<String,String>> stats = SaveLoadService.getPlayerStats();  
    // ordenat per GAMES_WON DESC, GAMES_PLAYED DESC  
    // per cada fila: nom, color (com a swatch), partides jugades, partides guanyades  
}
```

GameBoardController (~1780 línies)

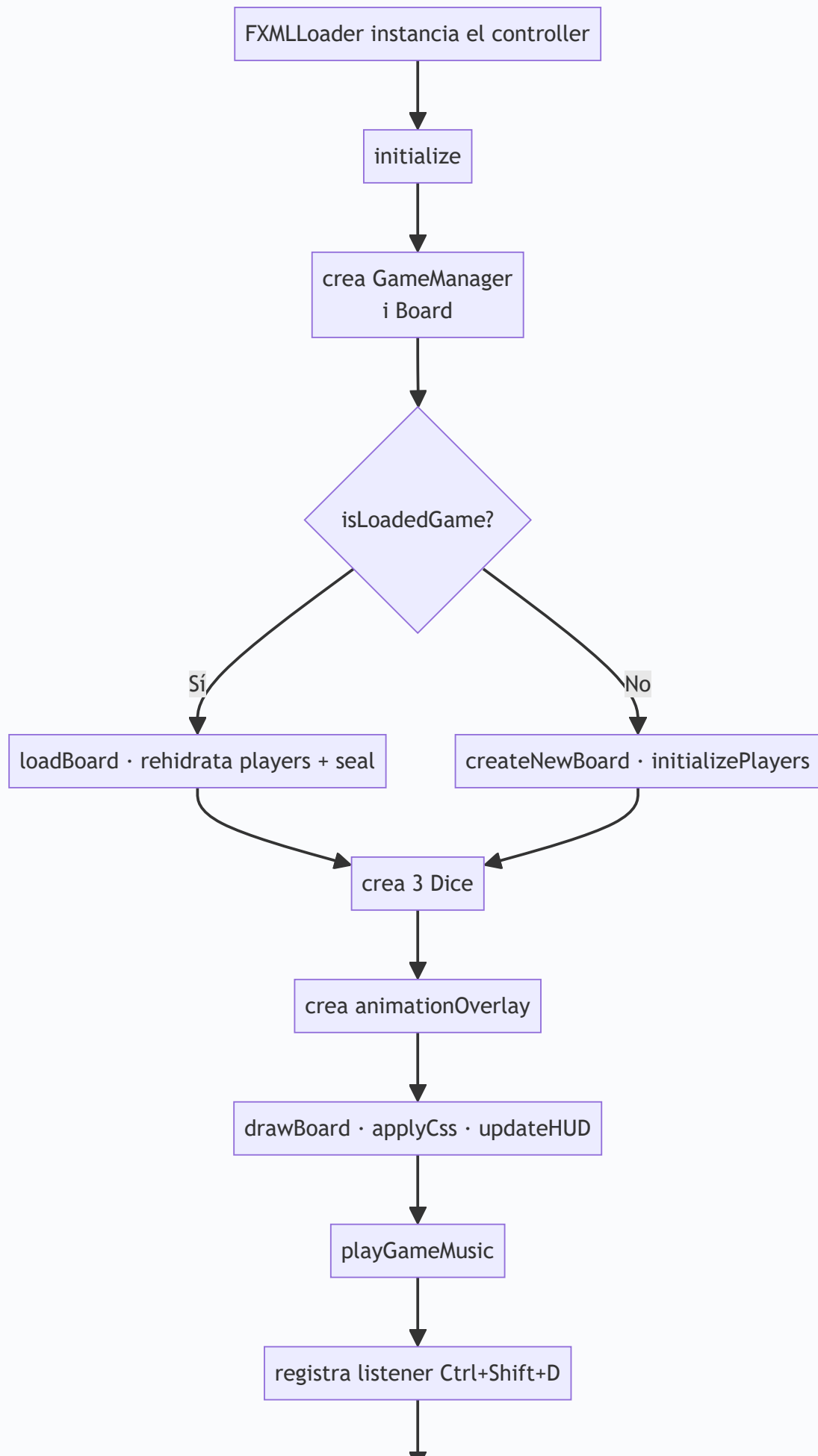
El controlador més gran. Gestiona tota la pantalla del joc.

Estat intern

```
private Board gameBoard;  
private TurnController turnController;  
private GameManager gameManager;  
private Dice defaultDice, fastDice, slowDice;  
private Seal seal;  
private boolean sealEnabled, gameOver;  
private StackPane animationOverlay;  
private boolean debugMode = false;  
private Integer debugForcedDice = null;
```

I 10 sprites pre-carregats (4 idle + 2 damaged + 2 ice + 2 seal).

Flux d'inicialització



registra resize listener sobre boardContainer

Handlers @FXML

Mètode	Trigger	Conté
<code>rollDice()</code>	Botó 	<code>processDiceRoll(rollOrForce(defaultDice), "Normal")</code>
<code>rollFastDice()</code>	Botó  	Comprova inventari, gasta 1 FASTDICE, tira (5-10)
<code>rollSlowDice()</code>	Botó 	Comprova inventari, gasta 1 SLOWDICE, tira (1-3)
<code>throwSnowball()</code>	Botó 	<code>ChoiceDialog</code> per triar objectiu, després <code>throwSnowball()</code> del <code>PlayerManager</code>
<code>saveGame()</code>	Botó 	<code>TextInputDialog</code> → <code>SaveLoadService.saveGame()</code>
<code>showEventHistory()</code>	Botó 	Mostra el log de la partida
<code>handleBack()</code> / <code>handleReturnToMenu()</code>	Botons ◀ / 	<code>playTitleMusic()</code> + carrega <code>mainMenu.fxml</code>

Mètodes interns crítics

- `processDiceRoll(result, type)`: orquestra l'animació + crida `gameManager.playTurn()` + decideix la següent reacció (BEAR sound, ICE_HOLE freeze, etc.).
- `animatePlayerMovement(p, steps, onFinished)`: `Timeline` amb un keyframe per casella (250ms cadascun).
- `drawBoard()`: redibuixa el `GridPane` amb les caselles, sprites, foca. Té un guard `isRedrawing` per evitar re-entrada.
- `createCell(idx, size)`: crea una `StackPane` amb background canvas + foreground canvas + sprites.
- `renderPlayersOnCell(...)`: dibuixa el sprite tintat amb `Lighting` (veure [Sistema de Sprites](#)).
- `formatActionMessage(result)`: tradueix `ActionType` a missatge humà.
- `toggleDebugMode()`: activa/desactiva el mode debug (veure [Mode Debug Tècnic](#)).

Listeners registrats

```
// Redibuixar quan canvia la mida del contenidor
boardContainer.widthProperty().addListener(resizeListener);
boardContainer.heightProperty().addListener(resizeListener);

// Drecera Ctrl+Shift+D
mainStack.sceneProperty().addListener((obs, oldScene, newScene) -> {
    if (newScene != null) {
        newScene.addEventFilter(KeyEvent.KEY_PRESSED, ev -> {
            if (ev.isControlDown() && ev.isShiftDown() && ev.getCode() == KeyCode.D) {
                toggleDebugMode();
                ev.consume();
            }
        });
    }
});
```

Enllaços relacionats

- [Paquet view](#) — fitxers FXML
- [Paquet model.game](#) — `GameManager`, `BoardManager`, `PlayerManager`
- [Sistema de Sprites](#) — com es renderitzen els pingüins
- [Mode Debug Tècnic](#) — implementació del Ctrl+Shift+D
- [Flux de Joc](#) — seqüències detallades

Paquet view

Conté els arxius FXML que descriuen declarativament la UI, els CSS que els estilen, i una classe Java (`BBDDPanel`) que serveix com a script de proves contra Oracle.

Estructura

```
view/
├── ui/
│   └── BBDDPanel.java
└── fxml/
    ├── mainMenu.fxml      ← MainMenuController
    ├── playerSetup.fxml   ← PlayerSetupController
    ├── gameBoard.fxml     ← GameBoardController
    └── playerStats.fxml   ← PlayerStatsController
```

I els CSS associats:

```
assets/css/
├── style.css              ← menú + setup + estadístiques
└── gameBoardStyle.css    ← pantalla del joc
```

Fitxers FXML

`mainMenu.fxml`

Arrel: `StackPane#rootPane.mainmenu_bg`. Conté un `HBox` > `VBox` amb:

```
StackPane          (rootPane)
├── HBox alignment=CENTER
│   └── VBox alignment=CENTER spacing=15
│       ├── StackPane
│       │   ├── Text (titleShadowText, classe mainmenu_title_shadow)
│       │   └── Text (titleText, classe mainmenu_maintitle)
│       ├── Button (newGame_button, onAction=#handleNewGame)
│       ├── Button (loadGame_button, onAction=#handleLoadGame)
│       ├── Button (stats_button, onAction=#handleStats)
│       └── ComboBox (language_combobox)
```

Controller: `MainMenuController` (Paquet controller).

`playerSetup.fxml`

Arrel: `StackPane#rootPane.mainmenu_bg`. Estructura:

```

VBox alignment=TOP_CENTER spacing=16
├─ HBox.nav-bar      (backButton)
├─ Text              (titleText, "setup-screen-title")
├─ HBox              (numPlayersCombo + sealCheckBox + labels)
├─ ScrollPane
│   └─ VBox          (playersContainer – fitxes generades dinàmicament)
└─ HBox              (selectExistingButton + startGameButton)

```

Controller: **PlayerSetupController**.

gameBoard.fxml

Arrel: **StackPane#mainStack**. Conté un **BorderPane#rootPane**:

```

StackPane#mainStack
├─ BorderPane#rootPane
│   ├── top:      HBox.top-bar (game title + turn indicator)
│   ├── center: StackPane#boardContainer
│   │   └─ GridPane#grid.game-grid (10x5)
│   ├── right: VBox#rightPanel (player card + inventory + seal status)
│   └─ bottom: FlowPane.action-bar
│       ├── rollDiceButton
│       ├── rollFastDiceButton
│       ├── rollSlowDiceButton
│       ├── throwSnowballButton
│       ├── saveGameButton
│       ├── historyButton
│       ├── backButton
│       └─ homeButton

```

Controller: **GameBoardController**.

‘RIGHT’ OCULTAT EN RUNTIME

El **rightPanel** es deixa al FXML però **GameBoardController.updateHUD()** el fa invisible perquè la informació es renderitza a la *hotbar* dinàmica al **top**.

playerStats.fxml

Pantalla amb una **VBox#statsContainer** on **PlayerStatsController** afegeix files dinàmicament.

CSS

style.css

Estils per al menú, setup i estadístiques. Classes destacades:

- **mainmenu_bg** — fons del menú principal
- **mainmenu_maintitle** / **mainmenu_title_shadow** — text en capes
- **mainmenu_button_newgame** / **_loadgame** — botons grans
- **mainmenu_language_combo** — dropdown d'idioma
- **setup-card** / **setup-card-title** — fitxa de jugador

- `setup-screen-title` / `setup-control-label` / `setup-combo` / `setup-checkbox`
- `stats-row` / `stats-cell-name` / `stats-cell-colour` / etc.
- `nav-btn-back` / `nav-btn-home` — navegació

gameBoardStyle.css

Estils específics del tauler:

- `game-grid` / `square` — graella i caselles base
- `square-normal` / `square-ice-hole` / `square-sled` / etc. — un per a cada `SquareType`
- `hotbar` / `hotbar-portrait` / `hotbar-title` / `hotbar-title-shadow` / `hotbar-player-name`
- `inventory-slot` / `inventory-quantity`
- `dice-result-badge` — overlay del resultat del dau
- `btn-roll` / `btn-fast-dice` / `btn-slow-dice` / `btn-snowball` / `btn-save` / `btn-history`
- `top-bar` / `action-bar` / `right-panel` / `seal-card`

BBDDPanel

Classe utilitat (no és UI real, malgrat estar al paquet `view.ui`). Té un `main()` que:

1. Connecta a Oracle.
2. Fa INSERT, UPDATE, DELETE de prova sobre la taula `ENTITY` amb un usuari `PinguTest`.
3. Imprimeix els resultats per consola.

NOMÉS PER DEBUG

No es crida des del joc real. Es pot executar manualment des d'Eclipse per validar que la BBDD respon.

Recursos addicionals

Tipus	Localització
Sprites de jugador	<code>assets/sprites/entities/player/*.png</code>
Sprites de foca	<code>assets/sprites/entities/seal/*.png</code>
Fons de casella	<code>assets/sprites/squares/background/Square-{0..6}.png</code>
Foreground de casella	<code>assets/sprites/squares/foreground/{TYPE}.png</code>
Fons del menú	<code>assets/sprites/backgrounds/1.png</code>
Icones d'objecte	<code>assets/sprites/objects/{snowball,fish,fastdice,slowdice}.png</code>
Música	<code>assets/sounds/main_screen_music.wav</code> , <code>bg_music.wav</code>
SFX	<code>assets/sounds/{dice,event,bear,seal,snowball}.wav</code>
Fonts	<code>assets/font/pixel-game.regular.otf</code> + extrude + unicode
Traduccions	<code>assets/lang/{en,es,ca,fr,pt,ro,ar,ru,uk,ff,jp}.yaml</code>

Enllaços relacionats

- [Paquet controller](#) — qui connecta amb cada FXML
- [Sistema de Sprites](#) — com es dibuixen sobre les caselles
- [Paquet model.db](#) — `BDDPanel` test

Sistema de Sprites

El joc utilitza **pixel-art** (sprites de 17×19 píxels per al pingüí) renderitzat a `Canvas` amb *nearest-neighbour* per garantir píxels nítids a qualsevol mida.

Layering d'una casella

Cada casella és una `StackPane` apilant fins a **4 capes** de `Canvas`:



Syntax error in text
mermaid version 10.9.6

Codi rellevant a `GameBoardController.createCell()`:

```
Image cellBackground = loadImage(getBackgroundImagePath(squareIndex));
if (cellBackground != null) {
    Canvas bgCanvas = new Canvas(cellSize, cellSize);
    bgCanvas.getGraphicsContext2D().setImageSmoothing(false);
    bgCanvas.getGraphicsContext2D().drawImage(cellBackground, 0, 0, cellSize, cellSize);
    cell.getChildren().add(bgCanvas);
}
// ... mateix patró per al foreground i sprites
```

PER QUÈ CANVAS I NO IMAGEVIEW?

`ImageView.setSmooth(false)` no és fiable a totes les versions de JavaFX. Amb `Canvas.GraphicsContext.setImageSmoothing(false)` garantim *nearest-neighbour* a totes les plataformes.

Sprites del jugador

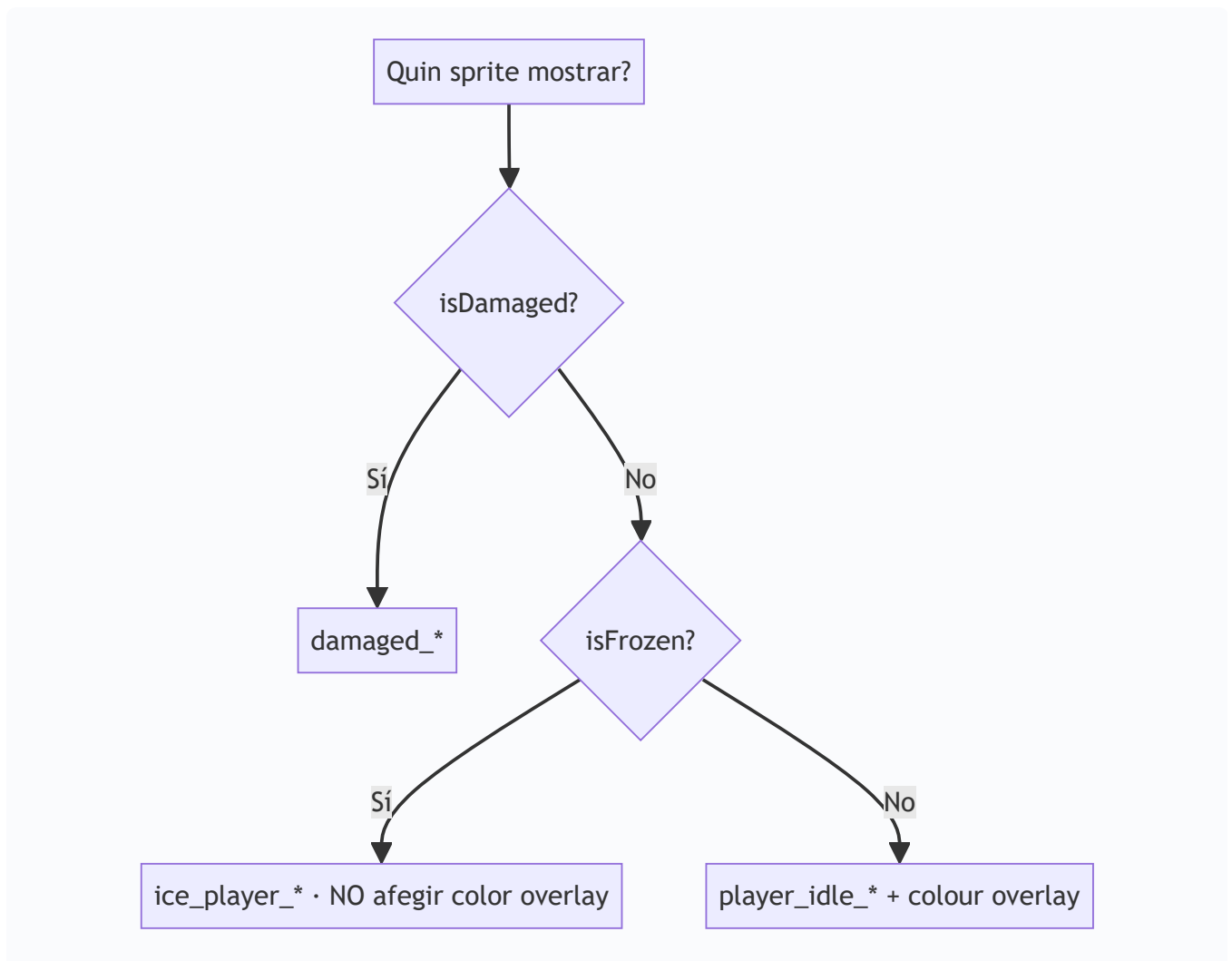
Per cada pingüí hi ha **8 sprites preparats** (carregats al constructor de `GameBoardController` i emmagatzemats en un `Map<String, Image> resourceCache`):

Sprite	Fitxer
Base idle dreta	player_idle_right.png
Base idle esquerra	player_idle_left.png
Color overlay dreta	player_idle_colour_right.png
Color overlay esquerra	player_idle_colour_left.png
Damaged dreta	player_damaged_right.png
Damaged esquerra	player_damaged_left.png
Frozen (ice) dreta	ice_player_right.png
Frozen (ice) esquerra	ice_player_left.png

I per la foca, 2 sprites (idle dreta/esquerra).

Selecció de sprite

Prioritat a `renderPlayersOnCell()` :



FROZEN NO TINTA

Quan el pingüí està congelat (`isFrozen==true`) NO s'afegeix la capa de tintat de color: el sprite de gel té el seu propi look i el tintat el pintaria a sobre o seria invisible.

Tintat de color amb `Lighting`

El sprite `player_idle_colour_*.png` és una **versió en escala de grisos** de la silueta. Per donar-li el color del jugador (RED/BLEU/etc.), s'usa l'efecte JavaFX `Lighting` amb un truc especial:

```
Lighting lighting = new Lighting(new Light.Distant(45, 90, getColorFromHex(player.getColour())));
lighting.setSurfaceScale(0.0); // 0 = sense relleu, només tintat
colorCanvas.setEffect(lighting);
```

SURFACESCALE = 0

Aquest valor desactiva el càlcul de profunditat del bump-mapping de `Lighting`, deixant només l'efecte de **multiplicar el color de la llum pel canal alfa**. Així obtenim un tintat pla i predictable.

Si no es pot carregar la imatge, hi ha fallback a un `Circle` del color del jugador.

Direcció de mirada

Es decideix segons la **fila** del tauler (snake pattern):

```
int row = squareIndex / Board.widthBoard;
boolean rowFacesRight = (row % 2 == 0);
Image baseSprite = rowFacesRight ? baseRightImage : baseLeftImage;
```

FILES PARELLS (0, 2, 4)

Els pingüins miren a la **dreta** (la serpent es mou cap a la dreta).

FILES SENARS (1, 3)

Els pingüins miren a l'**esquerra** (la serpent es mou cap a l'esquerra).

Apilament de múltiples jugadors

Si diversos pingüins ocupen la mateixa casella, es separen horitzontalment perquè es vegin tots:

```
double spacingVal = cellSize * 0.15;
playerToken.setTranslateX((idx - (count - 1) / 2.0) * spacingVal);
```

Backgrounds segons posició

A `getBackgroundImagePath(squareIndex)`:

Posició	Sprite
Casella 0 (START)	Square-0.png
Casella 49 (END)	Square-6.png
Cantonada esquerra fila parell	Square-5.png
Cantonada esquerra fila senar	Square-4.png
Cantonada dreta fila parell	Square-3.png
Cantonada dreta fila senar	Square-2.png
Casella central	Square-1.png

Foreground (només si no és NORMAL):

```
return loadImage("/assets/sprites/squares/foreground/" + type.name() + ".png");
// → BEAR.png, ICE_HOLE.png, SLED.png, EVENT.png, BROKEN_FLOOR.png, START.png, END.png
```

Recalcular mides

`drawBoard()` calcula la mida de cel·la a partir del contenidor:

```
double cellSize = Math.floor(Math.min(availW / cols, availH / rows));
```

I s'invoca cada vegada que canvia la mida del `boardContainer` gràcies a aquest listener:

```
ChangeListener<Number> resizeListener = (obs, oldVal, newVal) -> {
    if (newVal != null && newVal.doubleValue() > 0) drawBoard();
};
boardContainer.widthProperty().addListener(resizeListener);
boardContainer.heightProperty().addListener(resizeListener);
```

Un *flag* `isRedrawing` evita la re-entrada quan el redibuix dispara nous canvis de mida.

Animacions associades

Esdeveniment	Animació
Moviment de pingüí	Timeline amb 1 keyframe per casella (250ms) — <code>animatePlayerMovement</code>
Cop (snowball, bear, foca)	<code>flashDamage</code> — marca <code>damaged=true</code> 450ms i restaura
Caigut a forat	<code>setFrozen(true)</code> — persisteix fins la pròxima tirada del propietari
Resultat del dau	<code>showDiceResultBadge</code> — fade+scale d'un Label sobreposat 1s
Animació de dau	<code>showDiceAnimation</code> — GIF rolling de ~1.5s

Enllaços relacionats

- Paquet controller — `GameBoardController` és qui ho fa tot

- [Tauler de Joc](#) — vista d'usuari del que es renderitza
- Paquet `model.entity` — flags `damaged` i `frozen` viuen a `Entity`

Sistema de So

Centralitzat al singleton `SoundManager` (Paquet `model.game`). Necessita el mòdul `javafx.media`.

Arquitectura



Syntax error in text
mermaid version 10.9.6

Recursos d'àudio

Fitxer	Tipus	Quan sona
<code>main_screen_music.wav</code>	Música (loop)	Menú principal, configuració, estadístiques
<code>bg_music.wav</code>	Música (loop)	Pantalla de joc
<code>dice.wav</code>	SFX	Quan acaba l'animació de tirada (1.5s després del clic)
<code>event.wav</code>	SFX	Quan es trepitja una casella EVENT
<code>bear.wav</code>	SFX	Quan ataca l'ós
<code>seal.wav</code>	SFX	Torn de la foca o trobada amb la foca
<code>snowball.wav</code>	SFX	En llançar una bola de neu o en guerra

Tots viuen a `/assets/sounds/`.

Diferència Media vs AudioClip

- **Música:** `MediaPlayer` (`Media` + `MediaPlayer`) — apte per a pistes llargues amb control de cicle i volum.
- **SFX:** `AudioClip` — baixa latència, ideal per sons curts disparats múltiples vegades.

Càrrega amb fallback

A l'inici, si el fitxer no existeix només es loga però **no es llança excepció**:


```
private AudioClip loadSound(String path) {
    try {
        URL url = getClass().getResource(path);
        if (url != null) return new AudioClip(url.toExternalForm());
    } catch (Exception e) {
        System.out.println("Could not load sound: " + path);
    }
    return null;
}
```

Després, els *play* són no-ops si el clip és `null`:

```
public void playDiceSound() { if (diceSound != null) diceSound.play(); }
```

ROBUSTESA

Si falten fitxers de so, el joc segueix funcionant sense àudio. Útil per a desplegaments parcials.

Canvi de música

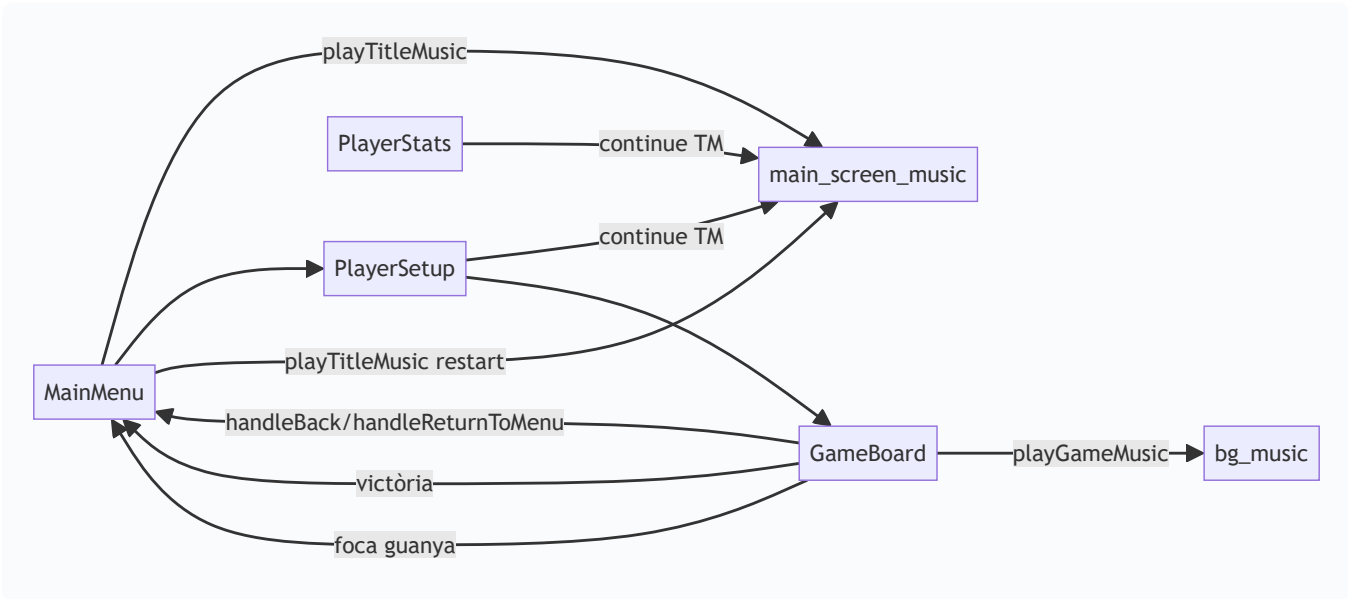
```
public void playTitleMusic() { switchMusic(titleMusicPlayer, gameMusicPlayer); }
public void playGameMusic() { switchMusic(gameMusicPlayer, titleMusicPlayer); }

private void switchMusic(MediaPlayer toPlay, MediaPlayer toStop) {
    if (toStop != null && toStop.getStatus() != Status.STOPPED) toStop.stop();
    if (toPlay != null) {
        toPlay.seek(Duration.ZERO); // sempre des del principi
        toPlay.play();
    }
}
```

REINICI DES DE ZERO

En tornar al menú principal des d'una partida, la música de títol **es reinicia des del començament** (no continua on s'havia parat). Mateixa cosa per a la música de joc en entrar a partida.

Cicle de música segons pantalla



Volum global

Constant `MUSIC_VOLUME = 0.40` aplicada a tots els `MediaPlayer`. Els `AudioClip` SFX van al volum natiu (no s'ajusta).

On es crida cada so

So	Crida des de	Quan
<code>playTitleMusic</code>	<code>MainMenuController.initialize</code> · <code>GameBoardController.handleBack/handleWin/handleReturnToMenu</code>	Tornar al menú
<code>playGameMusic</code>	<code>GameBoardController.initialize</code>	Començar partida
<code>playDiceSound</code>	<code>GameBoardController.processDiceRoll</code> (amb delay 1.5s)	Tirada del dau
<code>playEventSound</code>	<code>GameBoardController.runDiceMovement</code> (cas EVENT)	Casella EVENT
<code>playBearSound</code>	<code>GameBoardController.runDiceMovement</code> (cas BEAR_ATTACK)	Ós ataca
<code>playSealSound</code>	<code>GameBoardController</code> (turn foca / col·lisió)	Torn de la foca
<code>playSnowballSound</code>	<code>GameBoardController</code> (war / throw)	Llançar bola

Enllaços relacionats

- Paquet `model.game` — codi de `SoundManager`
- Paquet `controller` — qui crida els mètodes
- Tauler de Joc — pantalla amb música de joc

Sistema d'Idiomes

Localització basada en **YAML** + un singleton (`LangConfig`) + un enum (`Lang`) + el patró **Observer** per a refresc en viu.

Diagrama



Syntax error in text
mermaid version 10.9.6

El fitxer YAML

Format pla **clau:valor**, sense estructura jeràrquica (encara que les claus tinguin punts):

```
text.game.title: "El Joc del Pingüí"
menu.button.newgame: "Nova Partida"
menu.button.loadgame: "Carregar Partida"
gamesetup.player: "Jugador "
alert.wrongpassword.message: "La contrasenya del jugador '%s' és incorrecta..."
```

FORMAT PLA

SnakeYAML el carrega com a `Map<String, String>` on les claus mantenen els punts (`"menu.button.newgame"`). No s'utilitza estructura niada per simplicitat.

L'enum `Lang`

Cada constant Java guarda la seva clau YAML (consultable amb `.getKey()`):

```
public enum Lang {
    MENU_BUTTON_NEWGAME ("menu.button.newgame"),
    ALERT_WRONGPASSWORD_MESSAGE ("alert.wrongpassword.message"),
    // ... ~40 constants en total
    ;
    private final String key;
    Lang(String key) { this.key = key; }
    public String getKey() { return key; }
}
```

Lookup

```
public static String getLang(Lang lang) {
    LangConfig config = getInstance();
    String value = config.data.get(lang.getKey());
    return value != null ? value : lang.getKey();
}
```

FALLBACK ÚTIL

Si no hi ha traducció, retorna la **clau** (ex. "menu.button.newgame") en lloc de `null` . Fa que els missatges no traduïts siguin visibles a la UI sense crashejar.

Idiomes disponibles

Definits a l'array `AVAILABLE_LANGUAGES` :

```
private static final String[] AVAILABLE_LANGUAGES = {
    "en", "es", "ca", "fr", "pt", "ro", "ar", "ru", "uk", "ff", "jp"
};
```

I un `LinkedHashMap` (per preservar l'ordre del dropdown) amb les etiquetes mostrades a l'usuari (veure [Idiomes i Localització](#)).

Patró Listener

Cada controller que té text traduïble registra un callback al `LangConfig` :

```
@FXML
public void initialize() {
    LangConfig.addLanguageChangeListener(this::refreshTexts);
    refreshTexts(); // setup inicial
}

private void refreshTexts() {
    titleText.setText(LangConfig.getLang(Lang.TEXT_GAME_TITLE));
    newGame_button.setText(LangConfig.getLang(Lang.MENU_BUTTON_NEWGAME));
    // ...
}
```

I a `LangConfig.loadLang(code)` :

```
private void internalLoad(String langCode) {
    // ... carrega YAML
    data = yaml.load(inputStream);
    currentLang = langCode;
    notifyListeners();    // tots els controllers refresquen alhora
}

private static void notifyListeners() {
    for (Runnable listener : listeners) listener.run();
}
```

Quan s'inicialitza

A `MainMenu.main()`:

```
public static void main(String[] args) {
    LangConfig.loadLang();    // carrega "en.yml" per defecte
    launch(args);
}
```

I `loadLang()` (sense paràmetres) crida `loadLang(DEFAULT_LANG = "en")`.

Fallback de càrrega

```
private void internalLoad(String langCode) {
    try (InputStream in = getClass().getResourceAsStream(LANG_DIR + langCode + ".yml")) {
        if (in == null) {
            if (!langCode.equals(DEFAULT_LANG)) {
                internalLoad(DEFAULT_LANG);    // intenta en.yml
            }
        } else {
            data = yaml.load(in);
            currentLang = langCode;
            notifyListeners();
        }
    } catch (Exception e) { e.printStackTrace(); }
}
```

DOBLE FALLBACK INFINIT?

No, perquè el bloc `if (!langCode.equals(DEFAULT_LANG))` només es dispara una vegada. Si tampoc existeix `en.yml`, la UI queda sense traduir però **no entra en bucle**.

Mètodes auxiliars

Mètode	Funció
<code>getCurrentLang()</code>	Codi actiu actual (e.g. "ca")
<code>getAvailableLanguages()</code>	Array de codis
<code>getDisplayName(code)</code>	"🌐 Català" per "ca"
<code>cycleLanguage()</code>	Rotació al següent idioma (per a un botó simple sense ComboBox)
<code>removeLanguageChangeListener(r)</code>	Per netejar listeners quan es destrueix una pantalla

Comportament en canvi de pantalla

Quan canvis de pantalla els controllers antics encara estan registrats com a listeners (mai es desregistren explícitament al projecte actual). En la pràctica no és problema perquè els seus components ja no estan visibles. Per a una refactorització més neta caldria invocar `removeLanguageChangeListener` als `handleBack`.

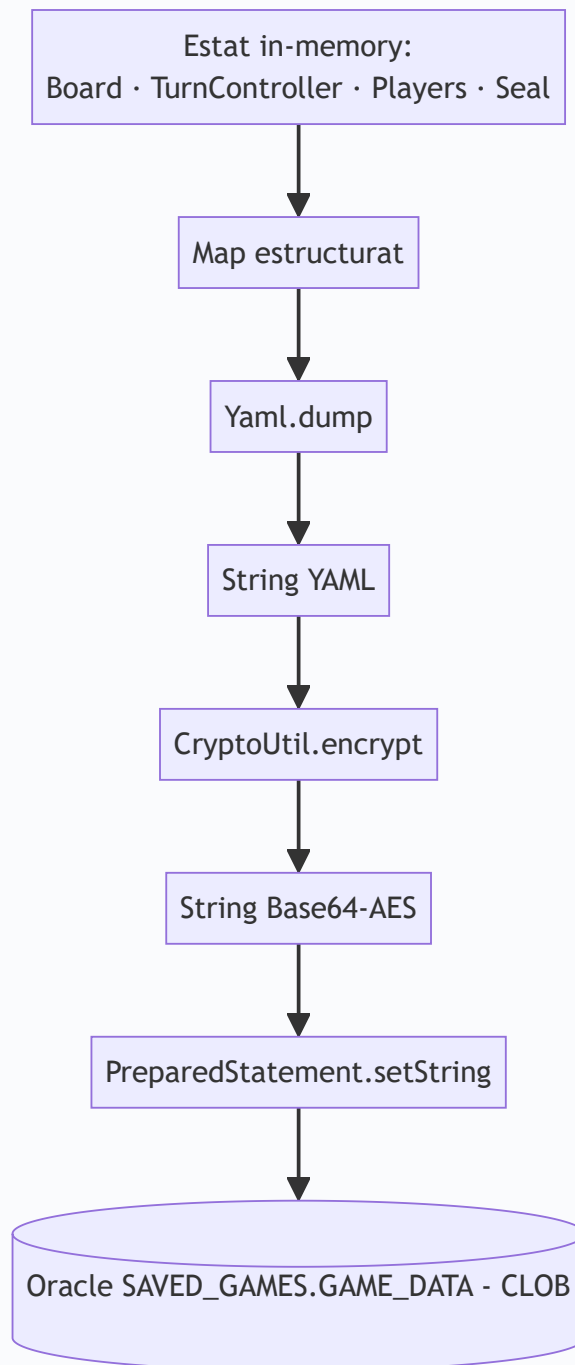
Enllaços relacionats

- Idiomes i Localització — vista d'usuari
- Paquet `model.config` — `Lang` i `LangConfig`
- Paquet `view` — fitxers YAML

Sistema de Persistència

Pipeline complet de guardat i càrrega de partides, jugadors i estadístiques. Combina **SnakeYAML** (serialització) + **AES-128 / Base64** (encriptació) + **Oracle JDBC** (storage).

Flux de guardat



Mètode `SaveLoadService.saveGame(...)`

Construeix un `Map<String, Object>` amb la informació següent:

```
Map<String, Object> state = new HashMap<>();
state.put("board", List<String>);           // SquareType.name() per cada casella
state.put("currentTurn", int);
state.put("players", List<Map>);           // veure sota
state.put("seal", Map);                     // opcional (square, blockedTurns)
```

Cada jugador es mapeja com:

```
{
  name: "Pingu",
  color: "FF0000",
  password: "xxx",           // clar a YAML; serà encriptat dins el CLOB global
  id: 12345,
  square: 17,
  skipNextTurn: false,
  inventory: { snowballs: 3, fish: 1, fastdice: 0, slowdice: 1 },
  eventHistory: [ "...", "..." ]
}
```

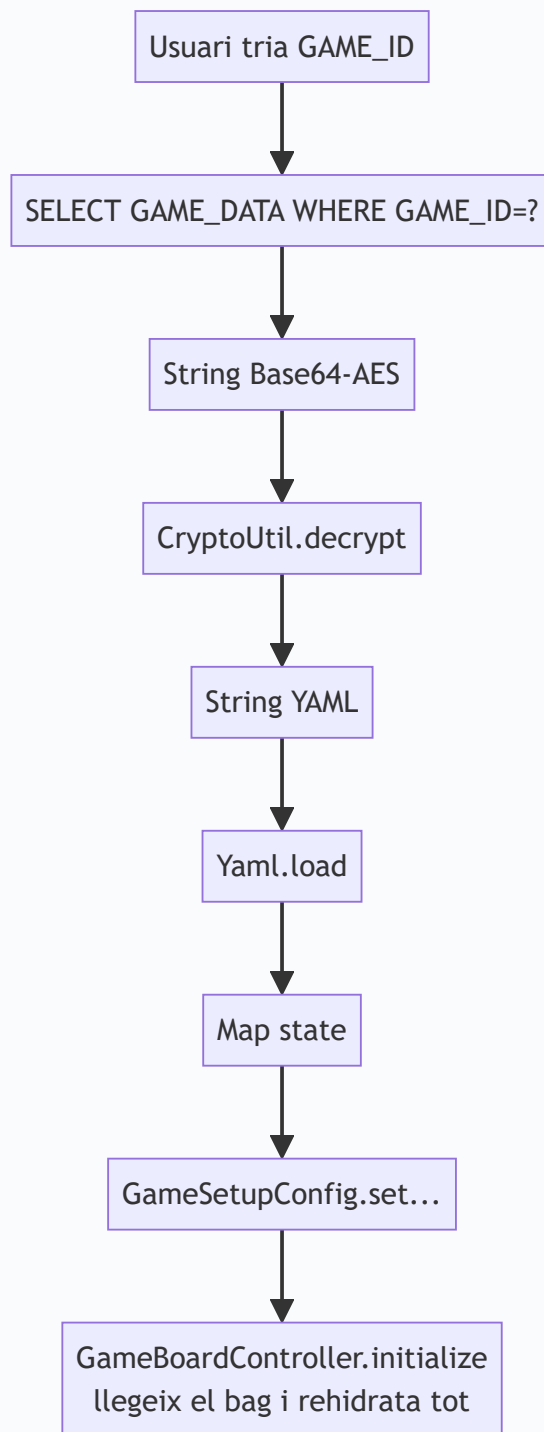
Després serialitza, encripta i insereix amb `PreparedStatement` :

```
String sql = "INSERT INTO SAVED_GAMES (GAME_ID, GAME_DATA) VALUES (?, ?)";
try (PreparedStatement ps = con.prepareStatement(sql)) {
  ps.setString(1, customName);
  ps.setString(2, encrypted);
  int rows = ps.executeUpdate();
  if (!con.getAutoCommit()) con.commit();
}
```

PER QUÈ `PREPAREDSTATEMENT` (I NO CONCATENACIÓ)?

La columna `GAME_DATA` és **CLOB** i l'YAML encriptat fàcilment supera els **4000 caràcters**, el límit dels literals SQL d'Oracle (`ORA-01704: string literal too long`). El `PreparedStatement` envia el valor en streaming via JDBC i evita el problema.

Flux de càrrega



`SaveLoadService.loadGame(gameId)` retorna `true/false` i deixa la informació a `GameSetupConfig`:

```

GameSetupConfig.setLoadedBoardState((List<String>) state.get("board"));
GameSetupConfig.setLoadedTurnIndex(((Number) state.get("currentTurn")).intValue());
GameSetupConfig.setPlayers(reconstructedPlayers);
if (state.containsKey("seal")) {
    GameSetupConfig.setSealEnabled(true);
    GameSetupConfig.setLoadedSealState((Map) state.get("seal"));
}
GameSetupConfig.setLoadedGame(true);

```

I `GameBoardController.initialize()` detecta `isLoadedGame()` i fa `gameBoard.loadBoard(...)` en lloc de `createNewBoard()`.

Encriptació amb `CryptoUtil`

```

private static final String ALGORITHM = "AES";
private static final byte[] KEY = "PinguGameKey1234".getBytes(); // 16 bytes

public static String encrypt(String value) {
    SecretKeySpec sk = new SecretKeySpec(KEY, ALGORITHM);
    Cipher cipher = Cipher.getInstance(ALGORITHM);
    cipher.init(Cipher.ENCRYPT_MODE, sk);
    byte[] enc = cipher.doFinal(value.getBytes("UTF-8"));
    return Base64.getEncoder().encodeToString(enc);
}

```

AES/ECB SENSE IV

`Cipher.getInstance("AES")` resol per defecte a `AES/ECB/PKCS5Padding` al JCE estàndard. Sense IV i sense MAC, el sistema **no és apte per a producció**. Per al projecte didàctic és suficient per ofuscar contrasenyes contra inspecció casual.

Registre de jugador

`SaveLoadService.registerPlayer(name, password, color)` usa una sentència **MERGE** d'Oracle per fer un *upsert* atòmic:

```

MERGE INTO ENTITY e
USING (SELECT 'Pingu' AS pname FROM DUAL) src
ON (e.PLAYERNAME = src.pname AND e.ENTITYTYPE = 'PLAYER')
WHEN MATCHED THEN
    UPDATE SET e.PLAYERPASSWORD = '<encrypted>', e.COLOUR = '<hex>'
WHEN NOT MATCHED THEN
    INSERT (ENTITYID, ENTITYTYPE, PLAYERNAME, PLAYERPASSWORD, COLOUR, GAMES_PLAYED, GAMES_WON)
    VALUES (<newId>, 'PLAYER', '<name>', '<encrypted>', '<hex>', 0, 0)

```

GENERACIÓ D'ENTITYID

En no usar `SEQUENCE/IDENTITY`, l'ID es deriva amb `SELECT NVL(MAX(ENTITYID), 0) + 1 FROM ENTITY`. Pot tenir condicions de carrera amb concurrència alta — acceptable per ús local.

Verificació de contrasenya

```
public static boolean verifyPassword(String playerName, String inputPassword) {
    String stored = SELECT PLAYERPASSWORD FROM ENTITY WHERE PLAYERNAME = ...
    if (stored == null || stored.isEmpty()) {
        return (inputPassword == null || inputPassword.isEmpty());
    }
    String decrypted = CryptoUtil.decrypt(stored);
    return inputPassword.equals(decrypted);
}
```

Si el jugador **no existeix** a la BBDD encara, retorna `true` (es considera una alta nova).

Estadístiques de partida

`recordGameResult(allPlayers, winnerName):`

1. Crea una fila a `BOARD` (`id=1`) si no existeix.
2. INSERT a `GAME` (`GAMESTATE='FINISHED', GAMEDATE=SYSDATE, BOARDID=1`).
3. Per cada jugador: `UPDATE ENTITY SET GAMES_PLAYED = NVL(GAMES_PLAYED, 0) + 1 WHERE PLAYERNAME='...'`.
4. Si hi ha guanyador: `UPDATE ENTITY SET GAMES_WON = NVL(GAMES_WON, 0) + 1 WHERE PLAYERNAME='<winner>'`.

PER QUÈ `NVL`?

A Oracle, `NULL + 1 = NULL`. Sense `NVL(col, 0)` les columnes podrien quedar a `NULL` eternament. `NVL` substitueix `NULL` per `0` abans de sumar.

Estructura de fitxer YAML d'exemple

```
board:
- START
- NORMAL
- BEAR
- NORMAL
- ICE_HOLE
- ... # 50 valors total acabant amb END
currentTurn: 1
players:
- name: Pingu
  color: FF0000
  password: hunter2
  id: 12345
  square: 17
  skipNextTurn: false
  inventory:
    snowballs: 3
    fish: 1
    fastdice: 0
    slowdice: 1
  eventHistory:
    - "🐧 4 (Normal) → 🐧 ..."
seal:
  square: 22
  blockedTurns: 0
```

Tot això s'encrypta abans de pujar-se a `SAVED_GAMES.GAME_DATA`.

Enllaços relacionats

- [Guardar i Carregar](#) — vista d'usuari
- [Paquet model.game](#) — codi de `SaveLoadService`
- [Paquet model.db](#) — capa Oracle (BBDD)
- [Paquet model.config](#) — `CryptoUtil` i `GameSetupConfig`

Diagrama de Classes

UML general (simplificat) de les classes principals del joc. Per veure detalls per paquet, consulta les pàgines individuals.

Visió completa



Syntax error in text
mermaid version 10.9.6

Enums



Syntax error in text
mermaid version 10.9.6

Patrons identificats

Patró	Implementació
MVC	view/, controller/, model/
Singleton	LangConfig, SoundManager
Template Method	Square abstracta amb action() redefinit per subclasses
Strategy via switch	BoardManager.executeSquareAction triant handler per SquareType
Observer / Listener	LangConfig.listeners notifica controllers en canviar idioma
DTO	ActionResult viatja de motor a UI
Static Configuration Bag	GameSetupConfig
Lazy initialization	SoundManager.getInstance(), LangConfig.getInstance()

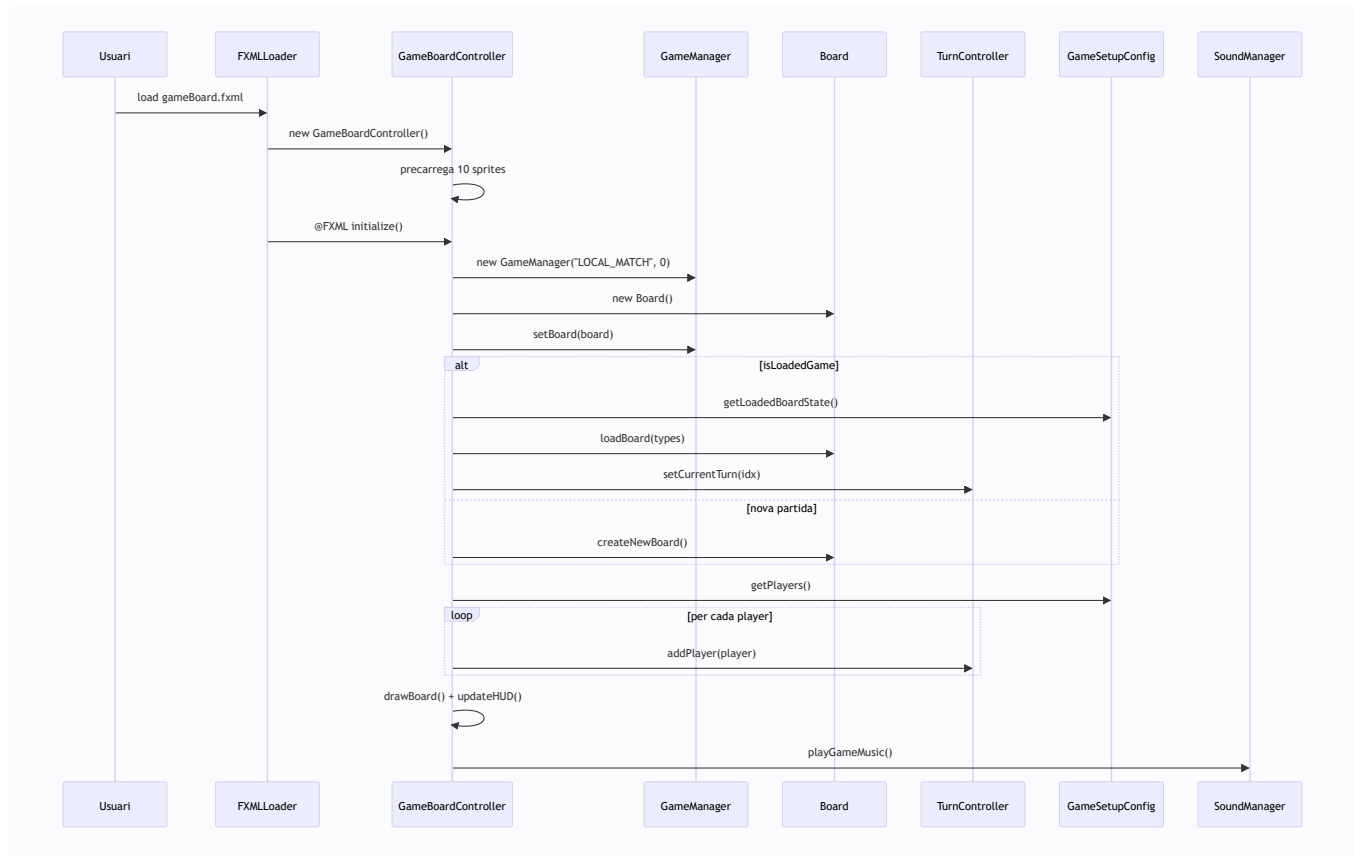
Enllaços relacionats

- Estructura de Paquets — mapping disc → classes
- Paquet model.board · Paquet model.entity · Paquet model.item · Paquet model.game
- Arquitectura General

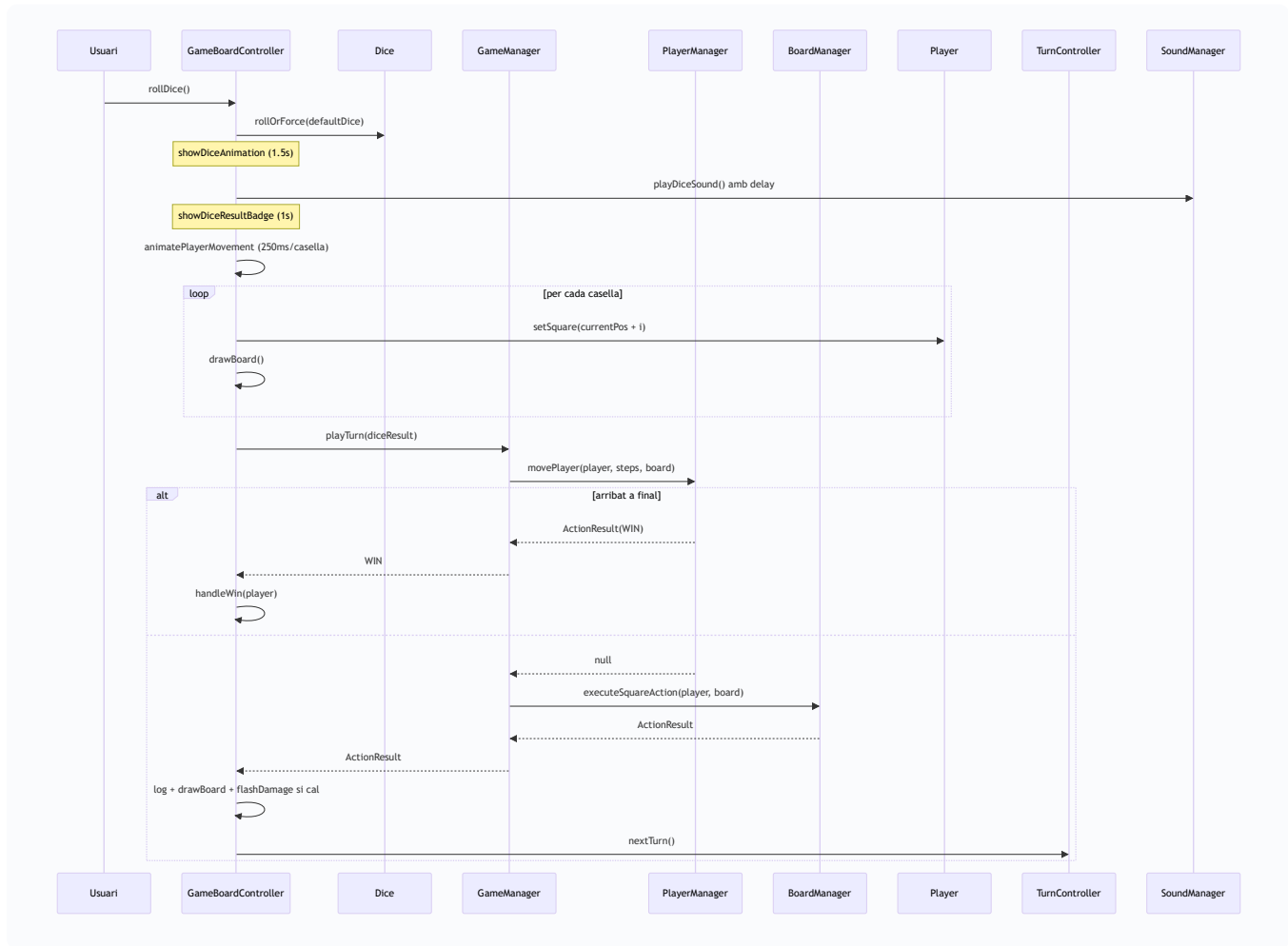
Flux de Joc

Diagrames de seqüència per a les operacions més importants del joc.

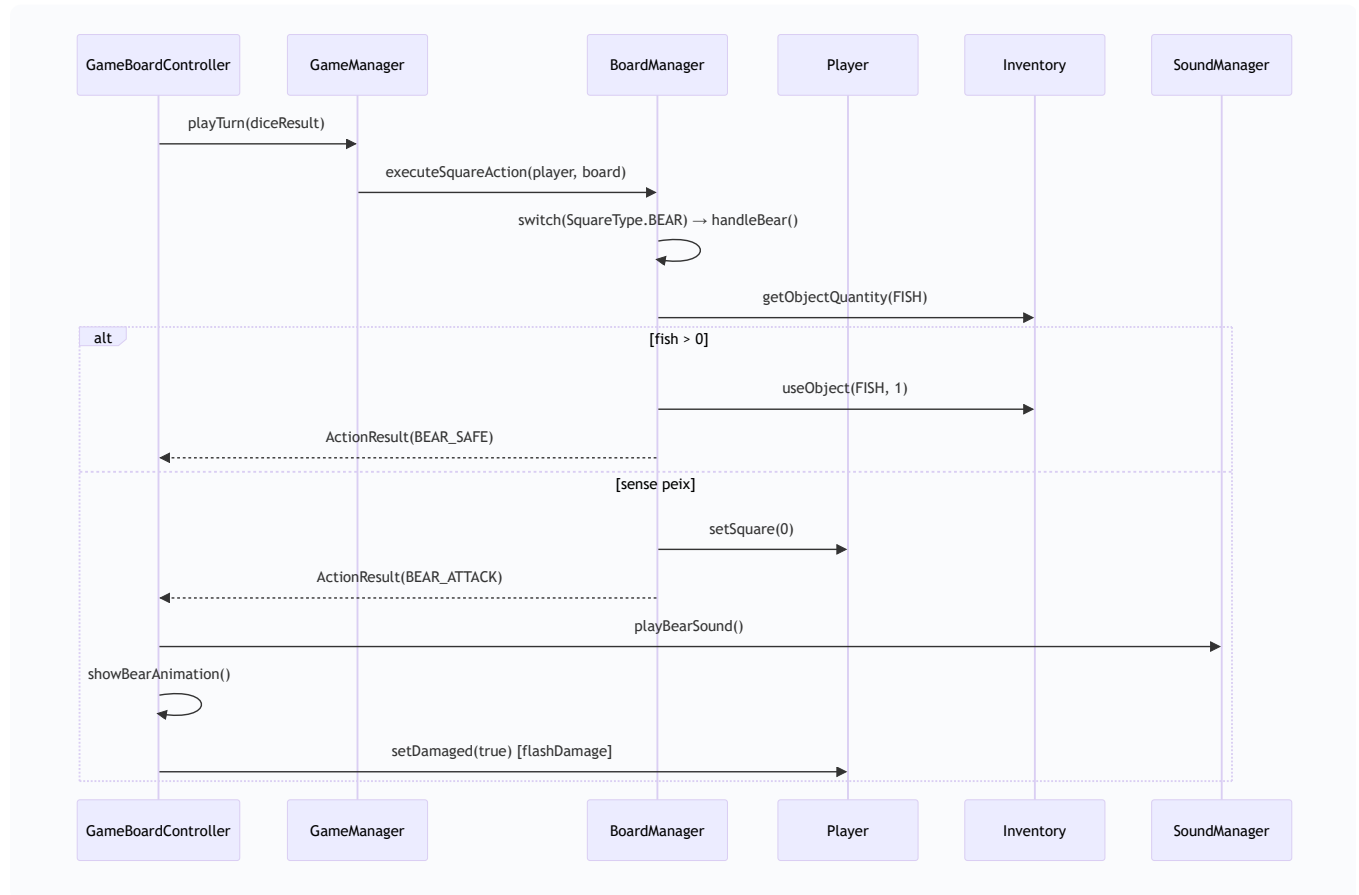
1. Inicialització de la partida



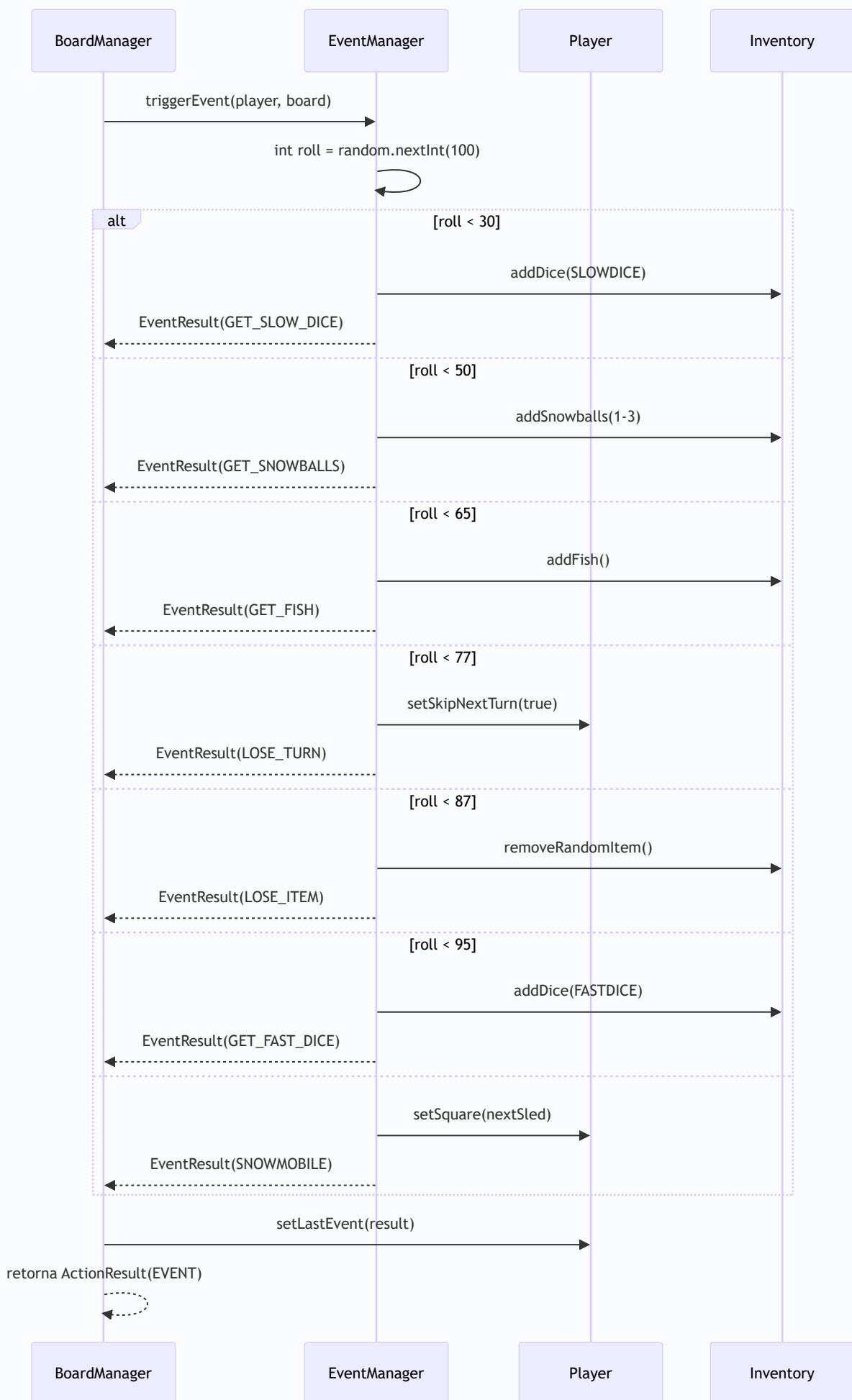
2. Tirada de daus i moviment



3. Caure en una casella especial (BEAR)



4. Casella EVENT amb subllançament aleatori



5. Torn de la foca CPU



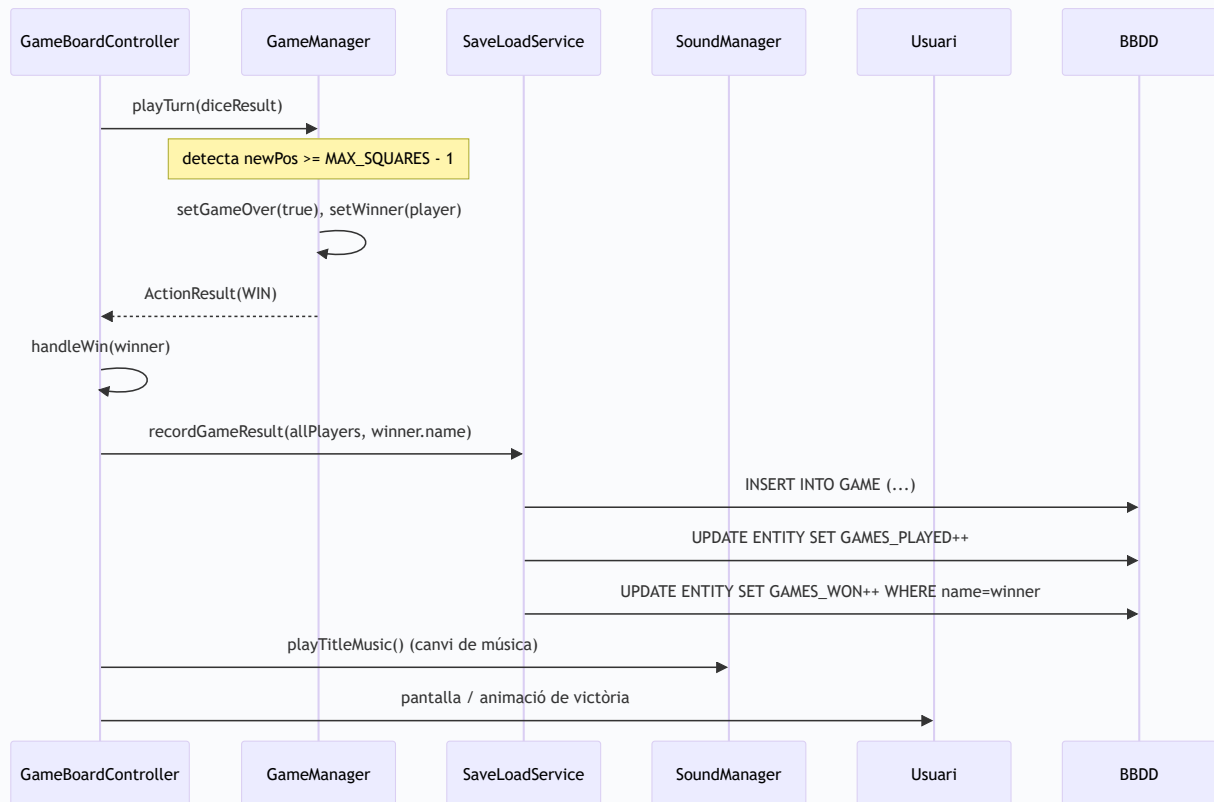
Syntax error in text
mermaid version 10.9.6

6. Snowball war (col·lisió de jugadors)

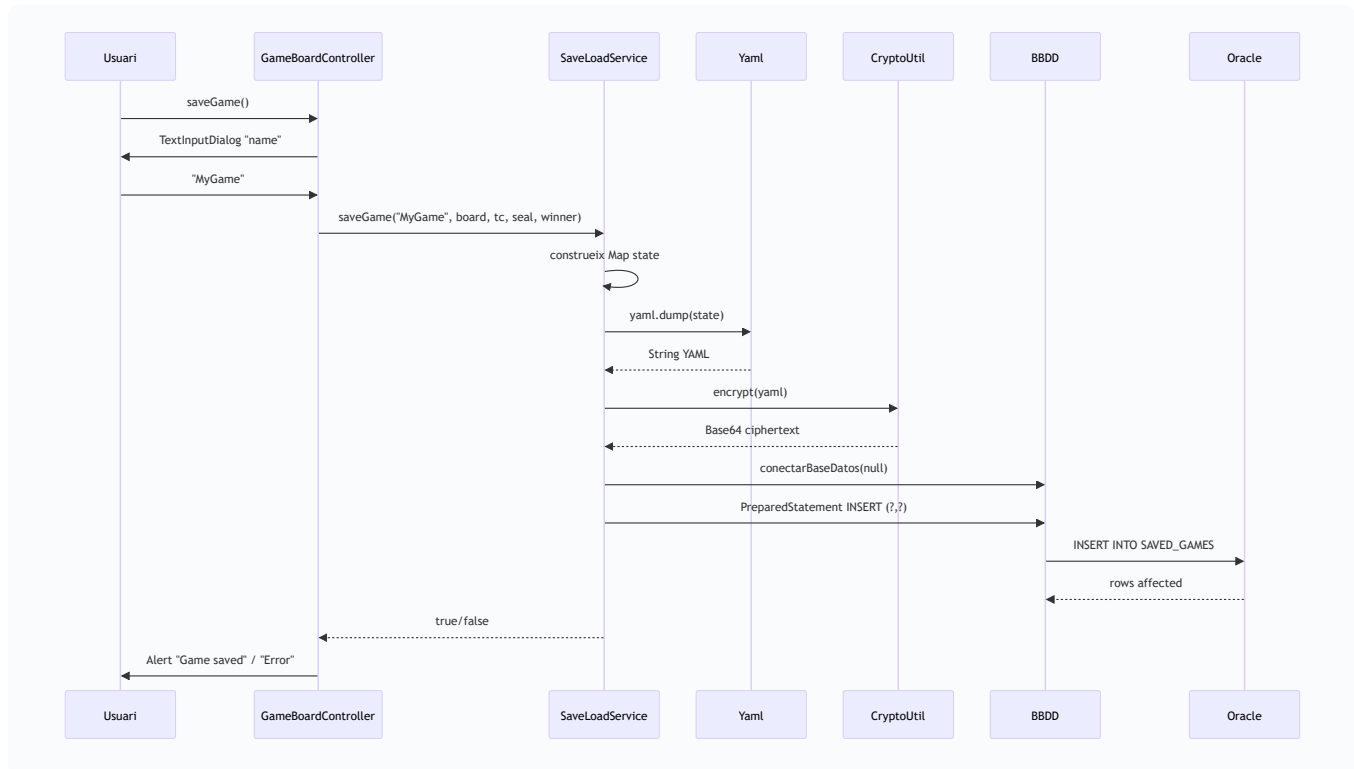


Syntax error in text
mermaid version 10.9.6

7. Victòria



8. Guardat de partida



9. Càrrega de partida



Syntax error in text
mermaid version 10.9.6

Enllaços relacionats

- Arquitectura General — visió MVC
- Paquet model.game — coordinadors
- Paquet controller — GameBoardController
- Caselles Especials — efecte de cada casella
- Sistema de Persistència — detalls del guardat

Mode Debug Tècnic

Documentació interna de la funcionalitat **Ctrl+Shift+D** del [Tauler de Joc](#). Codi tot al fitxer

`GameBoardController.java`.

Activació amb un Event Filter

L'avantatge dels *event filters* sobre els *handlers* convencionals és que **es disparen abans** del processament normal, permetent interceptar combinacions de tecles globalment sense haver d'estar a un widget concret amb focus.

```
mainStack.sceneProperty().addListener((obs, oldScene, newScene) -> {
    if (newScene != null) {
        newScene.addEventFilter(KeyEvent.KEY_PRESSED, ev -> {
            if (ev.isControlDown() && ev.isShiftDown() && ev.getCode() == KeyCode.D) {
                toggleDebugMode();
                ev.consume();
            }
        });
    }
});
```

PER QUÈ `SCENEPROPERTY().ADDLISTENER`?

Quan el controller s'inicialitza, l' `Scene` encara no està assignada al `Stage`. Esperem l'assignació via el listener i, llavors, registrem el filtre.

`toggleDebugMode()`

```
private void toggleDebugMode() {
    debugMode = !debugMode;
    if (debugPanel == null) buildDebugPanel();
    debugPanel.setVisible(debugMode);
    debugPanel.setManaged(debugMode);
    if (debugMode) {
        populateDebugPlayerCombo();
    } else {
        debugForcedDice = null;
        updateDebugBanner();
    }
    drawBoard(); // refresca els tokens per afegir/treure drag handlers
}
```

Construcció del panell

`buildDebugPanel()` crea un `VBox` amb 2 files i l'apila al `mainStack` amb alineació `TOP_CENTER`:

Row 1: ✖ DEBUG ON · Force next roll: [____] [Set] [Clear] · Drag any 🎲 to teleport.

Row 2: Inventory of: [combo ▼] · 🎲 N [+/·] · 🎲 N [+/·] · 🎲 🌟 N [+/·] · 🎲 N [+/·]

Estil inline (no usa el CSS general):

```
debugPanel.setStyle("-fx-background-color: rgba(0,0,0,0.85); -fx-padding: 6 14;");
StackPane.setAlignment(debugPanel, Pos.TOP_CENTER);
```

Forçar valor del dau

Variable `Integer debugForcedDice` (nullable):

```
private int rollOrForce(Dice dice) {
    if (debugForcedDice != null) {
        int v = debugForcedDice;
        debugForcedDice = null;    // un sol ús
        updateDebugBanner();
        return v;
    }
    return dice.roll();
}
```

Tots tres handlers (`rollDice`, `rollFastDice`, `rollSlowDice`) passen el dau a `rollOrForce()` perquè el valor forçat sobreescrigui qualsevol tipus de dau.

Rang acceptat al setter: **1-50** (perquè inclou tot el tauler).

```
int v = Integer.parseInt(diceField.getText().trim());
if (v >= 1 && v <= 50) {
    debugForcedDice = v;
    updateDebugBanner();
    diceField.clear();
}
```

Drag & drop dels pingüins

Cada vegada que `drawBoard()` renderitza un pingüí en mode debug, li afegeix tres handlers:

```
private void attachDebugDragHandlers(StackPane playerToken, Player player) {
    playerToken.setCursor(Cursor.OPEN_HAND);
    final double[] startTranslate = new double[2];
    final double[] startMouse = new double[2];

    playerToken.setOnMousePressed(e -> { ... });
    playerToken.setOnMouseDragged(e -> { ... });
    playerToken.setOnMouseReleased(e -> { ... });
}
```

Press

```
startTranslate[0] = playerToken.getTranslateX();
startTranslate[1] = playerToken.getTranslateY();
startMouse[0] = e.getSceneX();
startMouse[1] = e.getSceneY();
playerToken.setCursor(Cursor.CLOSED_HAND);

// Lift the parent cell so drag-over funciona per sobre de caselles posteriors
Node parentCell = playerToken.getParent();
if (parentCell != null) parentCell.toFront();
playerToken.toFront();
```

PER QUÈ `TOFRONT()`?

`GridPane` renderitza els fills en ordre d'inserció: les caselles amb índex més alt es dibuixen damunt. Si no portem la cel·la actual al davant, en arrossegar cap endavant l'sprite queda **tapat** per cel·les que encara no hem trepitjat.

Drag

Actualitza `translateX/Y` per moure visualment el sprite:

```
double dx = e.getSceneX() - startMouse[0];
double dy = e.getSceneY() - startMouse[1];
playerToken.setTranslateX(startTranslate[0] + dx);
playerToken.setTranslateY(startTranslate[1] + dy);
```

Release

```
int targetIndex = findCellIndexAt(e.getSceneX(), e.getSceneY());
if (targetIndex >= 0) {
    player.setSquare(targetIndex);
    logEvent("✂ DEBUG: " + player.getName() + " teleported to square " + targetIndex);
}
playerToken.setCursor(Cursor.OPEN_HAND);
drawBoard(); // redibuixa des de zero (això elimina els translates temporals)
```

Càlcul de l'índex de cel·la sota el cursor

`findCellIndexAt(sceneX, sceneY)` recorre els fills del grid:


```

int cols = Board.widthBoard;
for (Node child : grid.getChildren()) {
    Bounds b = child.localToScene(child.getBoundsInLocal());
    if (b.contains(sceneX, sceneY)) {
        Integer col = GridPane.getColumnIndex(child);
        Integer row = GridPane.getRowIndex(child);
        if (col != null && row != null) {
            return row * cols + (row % 2 == 0 ? col : cols - 1 - col);
        }
    }
}
return -1;

```

RECONVERSIÓ SNAKE → LINEAR

La fórmula $row * cols + (row \% 2 == 0 ? col : cols - 1 - col)$ reverteix el patró de serp del tauler: per a files senars, la columna `col` representa la posició a l'**inrevés** dins la fila.

Editor d'inventari

Per cada tipus d'objecte hi ha botons `+` i `-` connectats a `tweakInventory(type, delta)`:

```

private void tweakInventory(ObjectType type, int delta) {
    Player p = findDebugSelectedPlayer();
    if (p != null) {
        Inventory inv = p.getInventory();
        if (delta > 0) {
            switch (type) {
                case SNOWBALL: inv.addSnowballs(1); break;
                case FISH:     inv.addFish();      break;
                case FASTDICE: inv.addDice(ObjectType.FASTDICE); break;
                case SLOWDICE: inv.addDice(ObjectType.SLOWDICE); break;
                default: break;
            }
        } else if (inv.getObjectQuantity(type) > 0) {
            inv.useObject(type, 1);
        }
        refreshDebugInventoryLabels();
        updateHUD();
        drawBoard();
    }
}

```

RESPECTA ELS MÀXIMS

En afegir, es passa per `Inventory.addX()` que tenen els seus propis caps (`MAX_SNOWBALLS=6`, `MAX_FISH=2`, `MAX_DICE=3`). El debug **no fa cheats** contra aquests caps.

Etiquetes refrescades

`refreshDebugInventoryLabels()` actualitza els 4 labels d'inventari del panel sempre que canvia la selecció del combo o el contingut:

```
debugSnowballLabel.setText("❄️ " + inv.getSnowballQuantity() + "/" + Inventory.MAX_SNOWBALLS);
debugFishLabel.setText("🐟 " + inv.getFishQuantity() + "/" + Inventory.MAX_FISH);
debugFastDiceLabel.setText("🎲 ⚡ " + inv.getFastdiceQuantity());
debugSlowDiceLabel.setText("🎲 " + inv.getSlowdiceQuantity());
```

Persistència

EL MODE NO ES DESA

El flag `debugMode`, `debugForcedDice` i les configuracions del panell **no formen part del save**. En carregar una partida sempre es comença sense mode debug.

Enllaços relacionats

- [Mode Debug](#) — vista d'usuari
- [Paquet controller](#) — `GameBoardController`
- [Sistema de Sprites](#) — `drawBoard` és qui afegeix els handlers